



UNIVERSIDAD DE MÁLAGA



GRADO EN INGENIERÍA INFORMÁTICA

Aprendizaje por refuerzo para el control de alto nivel de robots móviles

Realizado por
David Solero Chicano

Tutorizado por
Ana María Cruz Martín y
Juan Antonio Fernández Madrigal

Departamento
(NOMBRE DEL DEPARTAMENTO)
UNIVERSIDAD DE MÁLAGA

MÁLAGA, (mes y 2026)



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUADO EN INGENIERÍA INFORMÁTICA

**Aprendizaje por refuerzo para el control de alto nivel de
robots móviles**

Reinforcement Learning for High-Level Control of Mobile Robots

Realizado por
David Solero Chicano

Tutorizado por
**Ana María Cruz Martín y
Juan Antonio Fernández Madrigal**

Departamento
[Nombre del departamento]

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2025

Fecha defensa:

Agradecimientos

Resumen

Este Trabajo de Fin de Grado aborda el problema del control de alto nivel en robots móviles mediante aprendizaje por refuerzo. El objetivo principal consistió en desarrollar un agente capaz de decidir de forma autónoma qué zonas visitar y cuándo acudir a una estación de carga, maximizando la cobertura del entorno y gestionando correctamente la autonomía energética del robot.

Para ello se diseñó un entorno de simulación en CoppeliaSim utilizando un robot Pioneer 3-DX, un modelo simplificado de navegación basado en grafos y un sistema de batería configurable. La integración entre el simulador y Python se realizó mediante la API ZeroMQ Remote, permitiendo definir el problema como un entorno compatible con Gymnasium y entrenar agentes mediante Stable-Baselines3. El algoritmo seleccionado fue PPO, debido a su estabilidad y adaptación natural a espacios de acciones discretos.

El trabajo se desarrolló siguiendo una metodología iterativa basada en el diseño progresivo de la función de recompensa y el análisis sistemático de los resultados obtenidos. Se realizaron dieciocho experimentos organizados en distintas líneas de investigación: recompensas densas, recompensas dispersas, ampliación del espacio de acciones e incorporación de información temporal en la observación del agente.

Los resultados muestran que el diseño de la función de recompensa tiene un impacto determinante sobre el comportamiento emergente del agente. Entre todos los experimentos realizados, exp_006 obtuvo el mejor equilibrio entre cobertura completa, estabilidad energética y balance entre habitaciones. Asimismo, se comprobó que la incorporación de información temporal modifica el comportamiento aprendido, aunque no resuelve completamente los problemas de balance observados en determinadas configuraciones.

Palabras clave: aprendizaje por refuerzo, robótica móvil, PPO, CoppeliaSim, control de alto nivel, diseño de la función de recompensa, control energético

Abstract

This Final Degree Project addressed the problem of high-level control in mobile robots through reinforcement learning. The main objective was to develop an autonomous agent capable of deciding which areas to visit and when to move to a charging station, maximizing environment coverage while efficiently managing the robot's battery autonomy.

To achieve this goal, a simulation environment was designed in CoppeliaSim using a Pioneer 3DX robot, a graph-based navigation model, and a configurable battery system. Communication between the simulator and Python was implemented through the ZeroMQ Remote API, allowing the problem to be defined as a Gymnasium-compatible environment and trained using Stable-Baselines3. PPO was selected as the reinforcement learning algorithm due to its stability and natural compatibility with discrete action spaces.

The project followed an iterative experimental methodology focused on the progressive design of the reward function and the systematic analysis of the obtained results. Eighteen experiments were conducted and grouped into several research lines: dense rewards, sparse rewards, action-space extension, and temporal information integration in the agent observations.

The results demonstrated that reward design has a decisive impact on the emergent behavior of the agent. Among all evaluated configurations, exp_006 achieved the best balance between full environment coverage, energy stability, and balanced room visitation. Furthermore, the inclusion of temporal information modified the learned policies, although it did not fully solve the balancing issues observed in some configurations.

Keywords: reinforcement learning, mobile robotics, PPO, CoppeliaSim, high-level control, reward function design, energy-aware control

Índice

Agradecimientos	1
Resumen	2
Abstract	1
Índice	1
Índice de Figuras	1
Introducción	1
1.1. Motivación	1
1.2. Objetivos	1
1.3. Estructura de la memoria	2
Estado del arte y tecnologías	3
2.1. Aprendizaje por refuerzo	3
2.2. Algoritmos de aprendizaje por refuerzo: de DQN a PPO	4
2.3. Gymnasium y Stable-Baselines3	5
2.4. CoppeliaSim como simulador robótico	5
2.5. Robótica de vigilancia y trabajos relacionados	6
Diseño del sistema	7
3.1. Arquitectura general	7
3.2. Entorno de simulación en CoppeliaSim	8
3.2.1. Escena y grafo de navegación	8
3.2.2. Script Lua: control del robot	9
3.2.3. Comunicación Python-CoppeliaSim	11
3.3. Entorno de aprendizaje por refuerzo	11
3.3.1. Espacio de observaciones	11
3.3.2. Espacio de acciones	11
3.3.3. Condiciones de finalización del episodio	12
3.3.4. Función de recompensa	12
3.4. Proceso de entrenamiento y evaluación	12
3.5. Gestión del tiempo en la simulación	12
3.5.1. Tipos de tiempo en el sistema	12
3.5.2. Aceleración de la simulación	13
3.5.3. Modo teleport	13
4.1. Enfoque iterativo	15
4.2. Criterios de evaluación	16
4.3. Estructura de experimentos	16
4.4. Herramientas de monitorización y análisis	16

Experimentos y resultados	18
5.1. Recompensa base: visitas equilibradas a habitaciones.....	19
5.1.1. exp_001 y exp_001_remake: recompensa por media de habitaciones	19
5.2. Incorporación de C en la media de visitas.....	20
5.2.1. exp_002: C en la media, sin recompensa por C	20
5.2.2. exp_003: C en la media, con recompensa por C.....	22
5.3. Penalización por visitar C.....	23
5.3.1. exp_004: penalización fija de -0.1 por ir a C	23
5.4. Penalización adaptativa y refinación del balance	24
5.4.1. exp_005: penalización adaptativa según nivel de batería	24
5.4.2. exp_006: penalización por sobrerrepresentación de habitación.....	26
5.4.3. exp_007: endurecimiento del umbral de penalización adaptativa.....	27
5.4.4. exp_008: penalización fija revisada.....	29
5.5. Recompensa basada en balance uniforme	30
5.5.1. exp_009: balance uniforme con penalización densa por C.....	30
5.6. Recompensa dispersa final.....	31
5.6.1. exp_010: recompensa dispersa de balance al final del episodio.....	31
5.6.2. exp_011: recompensa dispersa con penalización cuadrática de C (coeficiente 10)	33
5.6.3. exp_012: recompensa dispersa con penalización cuadrática de C (coeficiente 50)	34
5.7. Extensión del espacio de acciones: acción stop.....	36
5.7.1. exp_013: acción stop con velocidad acelerada	37
5.7.2. exp_014: acción stop con velocidad normal.....	38
5.8. Información temporal en el agente.....	40
5.8.1. exp_015: validación de SIM_ACCELERATION=3.0 (test de regresión)	41
5.8.2. exp_016: tiempo de episodio en la observación	42
5.8.3. exp_017: tiempo de episodio en observación y presión temporal en recompensa	44
5.8.4. exp_018: duración real de la última acción en la observación	45
5.9. Comparativa final	48
Conclusiones y líneas futuras	50
6.1. Conclusiones.....	50
6.2. Líneas futuras	51
Referencias.....	53
Apéndice A. Manual de Instalación	55
A.1. Requerimientos.....	55
A.2. Instalación y configuración	56
A.3. Ejecución básica	56
A.4. Reproducibilidad de experimentos	56
A.5. Solución de problemas comunes	57
A.6. Ubicación de datos y resultados	57
Apéndice B. Otros Recursos	58

Índice de Figuras

Fig. 1. Diagrama de sistema	8
Fig. 2. Entorno en Coppelia	9
Fig. 3. Modelo recarga batería	10
Fig. 4. Modelo de descarga de batería.....	10
Fig. 5.Ciclo de batería periódico	10
Fig. 6. Ejemplo de curva de Tensorflow	17
Fig. 7. Tiempo en C exp_001	20
Fig. 8. Balance entre habitaciones exp002	21
Fig. 9. Tiempo en C exp_002	21
Fig. 10. Tiempo en Carga exp_003.....	23
Fig. 11 Batería mínima exp_004	24
Fig. 12. Tiempo en C exp_005	25
Fig. 13. Probabilidad de recarga por batería previa exp_005.....	25
Fig. 14. Balance entre habitaciones exp_006	26
Fig. 15. Desbalance entre habitaciones exp_006	27
Fig. 16. Probabilidad de recarga por batería previa exp_007.....	28
Fig. 17. Batería mínima por episodio exp_007	28
Fig. 18. Balance entre habitaciones exp_008	29
Fig. 19. Balance entre habitaciones exp_009	31
Fig. 20. Balance entre habitaciones exp_010	32
Fig. 21. Probabilidad de recargar exp_011	33
Fig. 22. Tiempo en C por episodio	35
Fig. 23. Balance entre habitaciones exp_012	36
Fig. 24 Balance entre habitaciones exp_013	37
Fig. 25. Balance entre habitaciones exp_014	40
Fig. 26. Comparación curva exp_013 y exp_015	41
Fig. 28. Batería mínima exp_016	43
Fig. 27. Balance entre habitaciones exp_016	44
Fig. 29. Comparación primera recarga Exp_016 y Exp_017	45
Fig. 30. Balance entre habitaciones exp_018	47
Fig. 31. Batería mínima exp_018	47

1

Introducción

1.1. Motivación

En los últimos años, la robótica móvil ha experimentado un crecimiento notable en aplicaciones de vigilancia, inspección y monitorización de entornos interiores. En este contexto, uno de los problemas fundamentales es la toma de decisiones de alto nivel: determinar qué zonas visitar, en qué orden y cuándo interrumpir la misión para recargar la batería, todo ello de forma autónoma y sin intervención humana.

Los enfoques clásicos de planificación resuelven este problema mediante reglas fijas o heurísticas diseñadas manualmente, lo que limita su capacidad de adaptación ante condiciones cambiantes o no previstas. El aprendizaje por refuerzo (RL, del inglés Reinforcement Learning) ofrece una alternativa: permite que un agente aprenda una política de comportamiento óptima a través de la experiencia, sin necesidad de programar explícitamente cada caso. Esta característica lo convierte en un enfoque especialmente adecuado para problemas donde el comportamiento deseado es difícil de especificar a priori, pero fácil de evaluar mediante una función de recompensa.

El presente Trabajo de Fin de Grado aborda precisamente este problema desde la perspectiva del control de alto nivel: el agente no aprende a mover el robot ni a planificar trayectorias, sino a decidir qué destinos visitar y en qué orden, asumiendo que la navegación entre nodos ya está resuelta por la capa inferior del sistema. Esta separación de niveles es deliberada: permite aislar el problema de decisión estratégica y estudiarlo con rigor, sin que el ruido del control de bajo nivel interfiera en el análisis. Adicionalmente, el trabajo explora el efecto de incorporar información temporal en el agente, estudiando si el conocimiento del tiempo transcurrido o del tiempo consumido por cada acción influye en la calidad de las políticas aprendidas.

1.2. Objetivos

El objetivo principal de este trabajo es desarrollar un sistema de control de alto nivel para un robot móvil simulado, basado en aprendizaje por refuerzo, que sea capaz de

maximizar la cobertura del entorno gestionando de forma eficiente su autonomía energética.

Para alcanzar este objetivo, se plantean los siguientes objetivos específicos:

- Diseñar e implementar un entorno de simulación en CoppeliaSim con un robot Pioneer P3DX, un mapa de nodos navegables y un modelo de batería realista.
- Desarrollar una interfaz de comunicación entre el simulador y Python que permita ejecutar episodios de entrenamiento de forma programática y en modo sin interfaz gráfica (headless).
- Definir el problema de aprendizaje por refuerzo: espacio de estados, espacio de acciones y función de recompensa.
- Implementar y entrenar un agente mediante algoritmos de aprendizaje por refuerzo, utilizando la librería Stable-Baselines3.
- Evaluar el comportamiento aprendido y analizar los resultados obtenidos.
- Estudiar el efecto de la información temporal en el comportamiento del agente, analizando si incorporar señales de tiempo en la observación o en la función de recompensa mejora la eficiencia de las políticas aprendidas en tareas de control de alto nivel.

1.3. Estructura de la memoria

El resto de la memoria se organiza de la siguiente forma:

El capítulo 2 presenta el estado del arte, revisando los fundamentos del aprendizaje por refuerzo, los principales algoritmos empleados en robótica móvil y las herramientas utilizadas en el desarrollo.

El capítulo 3 describe el sistema desarrollado: el entorno de simulación en CoppeliaSim, la integración con Python mediante la API ZeroMQ y la definición del problema de aprendizaje por refuerzo como entorno Gymnasium.

El capítulo 4 describe la metodología de trabajo empleada: el ciclo iterativo de diseño, entrenamiento y evaluación, los criterios de éxito definidos y las herramientas de análisis utilizadas.

El capítulo 5 presenta los experimentos realizados y sus resultados, organizados en cuatro líneas de evolución: diseño de la función de recompensa, transición a recompensa dispersa, extensión del espacio de acciones e incorporación de información temporal.

El capítulo 6 expone las conclusiones del trabajo y las posibles líneas de trabajo futuro. Finalmente se incluyen los apéndices con información sobre la instalación del sistema y la estructura del repositorio.

2

Estado del arte y tecnologías

Este capítulo revisa los fundamentos teóricos y las herramientas sobre las que se asienta el trabajo. Se organiza en cuatro bloques: los principios del aprendizaje por refuerzo, el algoritmo PPO y su justificación frente a alternativas, el marco de trabajo Gymnasium y la librería Stable-Baselines3, y finalmente el simulador CoppeliaSim.

2.1. Aprendizaje por refuerzo

El aprendizaje por refuerzo (RL, del inglés Reinforcement Learning) es una rama del aprendizaje automático en la que un agente aprende a tomar decisiones mediante la interacción con un entorno. A diferencia del aprendizaje supervisado, en RL no existe un conjunto de ejemplos etiquetados: el agente recibe señales de recompensa como consecuencia de sus acciones y ajusta su comportamiento para maximizar la recompensa acumulada a lo largo del tiempo.

El marco formal que sustenta el RL es el Proceso de Decisión de Markov (MDP, del inglés Markov Decision Process), definido por la tupla (S, A, P, R, γ) , donde S es el espacio de estados, A el espacio de acciones, P la función de transición de estados, R la función de recompensa, y $\gamma \in [0,1)$ el factor de descuento que pondera la importancia de las recompensas futuras.

En cada paso de tiempo t , el agente observa el estado s_t , selecciona una acción a_t según su política $\pi(a|s)$, recibe una recompensa r_t y transita al nuevo estado s_{t+1} .

El objetivo del agente es encontrar una política óptima π^* que maximice el retorno esperado:

$$G_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \dots$$

La función de valor $V^\pi(s)$ expresa el retorno esperado desde el estado s siguiendo la política π . La función de valor de acción $Q^\pi(s,a)$ extiende este concepto a pares estado-acción. Las ecuaciones de Bellman describen la relación recursiva entre estos valores y constituyen la base teórica de la mayoría de algoritmos de RL.

Un concepto central es el dilema exploración-explotación: el agente debe equilibrar la exploración de acciones desconocidas, que pueden conducir a mejores recompensas, con la explotación de las acciones que ya sabe que funcionan bien. Este equilibrio es especialmente relevante en las fases iniciales del entrenamiento.

2.2. Algoritmos de aprendizaje por refuerzo: de DQN a PPO

Existen múltiples familias de algoritmos de RL. A continuación, se revisan brevemente los más relevantes para este trabajo, con el objetivo de justificar la elección de PPO.

DQN (Deep Q-Network) fue uno de los primeros algoritmos que combinó redes neuronales profundas con RL, aprendiendo directamente la función Q a partir de píxeles. Sin embargo, DQN está restringido a espacios de acciones discretos y tiende a ser inestable en entornos con recompensas densas y muchos pasos por episodio.

A2C/A3C (Advantage Actor-Critic) introduce la arquitectura actor-crítico: el actor aprende la política y el crítico estima el valor del estado para reducir la varianza del gradiente. Son algoritmos eficientes pero sensibles a los hiperparámetros y al tamaño del paso de aprendizaje.

SAC (Soft Actor-Critic) es un algoritmo off-policy basado en la maximización de entropía, muy eficiente en espacios de acciones continuos. Sin embargo, está diseñado para acciones continuas y no se adapta de forma natural a espacios discretos como el de este trabajo.

PPO (Proximal Policy Optimization), propuesto por Schulman et al. (2017), es actualmente uno de los algoritmos de RL más utilizados tanto en investigación como en aplicaciones prácticas. PPO es un algoritmo on-policy de tipo actor-crítico que introduce una restricción sobre la magnitud de la actualización de la política en cada paso. Esta restricción se implementa mediante una función objetivo recortada (clipped objective) que impide actualizaciones demasiado grandes, dotando al algoritmo de una notable estabilidad durante el entrenamiento.

Las principales razones por las que se eligió PPO en este trabajo son las siguientes.

- Primero, funciona de forma nativa con espacios de acciones discretos, que es exactamente el tipo de espacio que se define en este problema (cuatro destinos posibles).

- Segundo, es robusto frente a una amplia variedad de configuraciones de hiperparámetros, lo que resulta ventajoso en una fase experimental donde la función de recompensa evoluciona entre experimentos.
- Tercero, está disponible en implementaciones de alta calidad y bien documentadas, en particular en la librería Stable-Baselines3.
- Por último, su carácter on-policy garantiza que el agente aprende directamente de la política que está ejecutando en cada momento, lo cual favorece la coherencia entre el comportamiento explorado y el aprendido.

2.3. Gymnasium y Stable-Baselines3

Gymnasium es la librería estándar de facto para definir entornos de aprendizaje por refuerzo en Python. Desarrollada y mantenida por la Farama Foundation como evolución del antiguo OpenAI Gym, Gymnasium proporciona una interfaz común basada en tres métodos principales: `reset()`, que inicializa el episodio y devuelve la observación inicial; `step(action)`, que ejecuta una acción y devuelve la observación siguiente, la recompensa, y los flags de finalización; y `close()`, que libera los recursos del entorno.

Esta interfaz estandarizada permite que cualquier algoritmo de RL compatible con Gymnasium pueda operar sobre cualquier entorno que la implemente, sin modificaciones. En este trabajo, el entorno RobotEnv hereda de `gymnasium.Env` e implementa esta interfaz, lo que hace posible conectarlo directamente con Stable-Baselines3.

Stable-Baselines3 (SB3) es una librería Python que proporciona implementaciones limpias, fiables y bien documentadas de los principales algoritmos de RL modernos, incluyendo PPO, A2C, SAC y DQN. SB3 está construida sobre PyTorch, sigue las convenciones de Gymnasium y ofrece herramientas de integración con TensorBoard para la monitorización del entrenamiento. Su diseño modular permite personalizar la arquitectura de la red neuronal, los hiperparámetros y las estrategias de exploración con pocas líneas de código. En este trabajo se utiliza la implementación de PPO de SB3 con su política `MlpPolicy`, que corresponde a una red neuronal densa (Multi-Layer Perceptron).

2.4. CoppeliaSim como simulador robótico

CoppeliaSim (anteriormente conocido como V-REP) es un simulador robótico de propósito general desarrollado por Coppelia Robotics. Incorpora un motor de física integrado, soporte para múltiples modelos de robots y sensores, y un sistema de scripting en Lua que permite programar el comportamiento de los objetos directamente en la escena.

Entre sus características más relevantes para este trabajo destaca su API ZeroMQ Remote, que permite controlar la simulación desde Python mediante el paso de mensajes síncronos o asíncronos, y su modo de ejecución headless, que posibilita lanzar

simulaciones sin interfaz gráfica, reduciendo significativamente el consumo de recursos durante el entrenamiento.

En la versión educativa, CoppeliaSim Edu, es gratuita y ampliamente utilizada en entornos académicos, lo que la convierte en una opción idónea para proyectos de investigación y TFG. El robot seleccionado para las simulaciones es el Pioneer 3-DX, un robot diferencial de dos ruedas con amplia presencia en la literatura de robótica móvil, cuya dinámica y modelo de colisión están disponibles de serie en la biblioteca de CoppeliaSim.

La integración entre CoppeliaSim y Python se realiza mediante la librería `coppelasim_zmqremoteapi_client`, que expone la API del simulador como un objeto Python. Esta arquitectura permite que el entorno Gymnasium actúe como capa de control de alto nivel, enviando comandos de navegación al script Lua del robot y leyendo el estado resultante, todo ello de forma transparente para el algoritmo de RL.

2.5. Robótica de vigilancia y trabajos relacionados

La vigilancia autónoma mediante robots móviles es un problema ampliamente estudiado en la literatura. Los primeros trabajos se centraron en algoritmos deterministas de patrullaje, como los propuestos por Chevalleyre (2004), que definían rutas cíclicas para maximizar la frecuencia de visita a cada zona. Trabajos posteriores incorporaron planificación basada en grafos y heurísticas para entornos dinámicos.

La incorporación del aprendizaje por refuerzo a este dominio ha ganado interés en la última década. Investigaciones como las de Huang et al. (2019) demuestran que agentes entrenados con DQN o PPO son capaces de aprender políticas de patrullaje eficientes en entornos simulados, superando en equilibrio de cobertura a las soluciones basadas en reglas fijas. La gestión de la batería como recurso limitado ha sido abordada en trabajos como el de Notomista et al. (2020), que formulan el problema como un MDP con restricciones energéticas, obteniendo políticas que aprenden a recargar de forma proactiva antes de que la batería se agote.

Este trabajo se enmarca en esa línea de investigación, abordando simultáneamente el equilibrio de cobertura entre habitaciones y la gestión autónoma de la energía, en un entorno simulado de complejidad controlada que permite analizar con detalle el impacto de distintas funciones de recompensa sobre el comportamiento emergente del agente.

3

Diseño del sistema

Este capítulo describe el sistema desarrollado: la capa de simulación en CoppeliaSim, la capa de control en Python y la definición del problema de aprendizaje por refuerzo.

3.1. Arquitectura general

El sistema se organiza en tres niveles que se comunican de forma síncrona.

El nivel más bajo es la capa de simulación, implementada en Lua dentro de CoppeliaSim. Controla la cinemática del robot Pioneer P3DX, el modelo de batería y el grafo de rutas. Expone su estado mediante señales ZeroMQ y acepta comandos en forma de texto.

El nivel intermedio es la capa de entorno RL, implementada en Python a través de RobotEnv (hereda de gymnasium.Env). Traduce acciones discretas del agente en comandos Lua, lee el estado resultante y calcula la recompensa.

El nivel más alto es la capa de aprendizaje, donde opera PPO de Stable-Baselines3. Interactúa exclusivamente con RobotEnv a través de la interfaz Gymnasium.

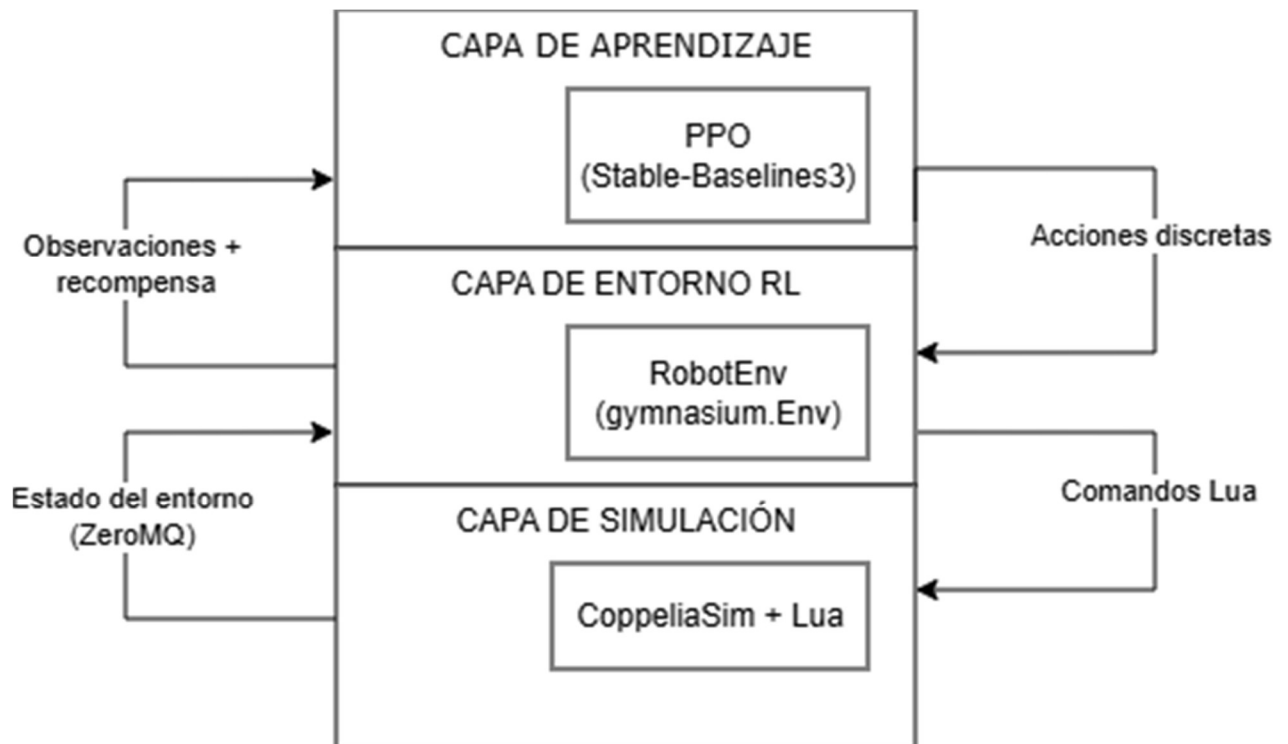


Fig. 1. Diagrama de sistema

3.2. Entorno de simulación en CoppeliaSim

3.2.1. Escena y grafo de navegación

La escena representa un entorno interior compuesto por cinco nodos navegables: el nodo de inicio R, tres habitaciones Hab1, Hab2 y Hab3, y una estación de carga C. Cada nodo está conectado con el resto mediante rutas bidireccionales predefinidas, formando un grafo completamente conexo. Las rutas se definen como objetos Path y el robot las sigue mediante interpolación de posición.

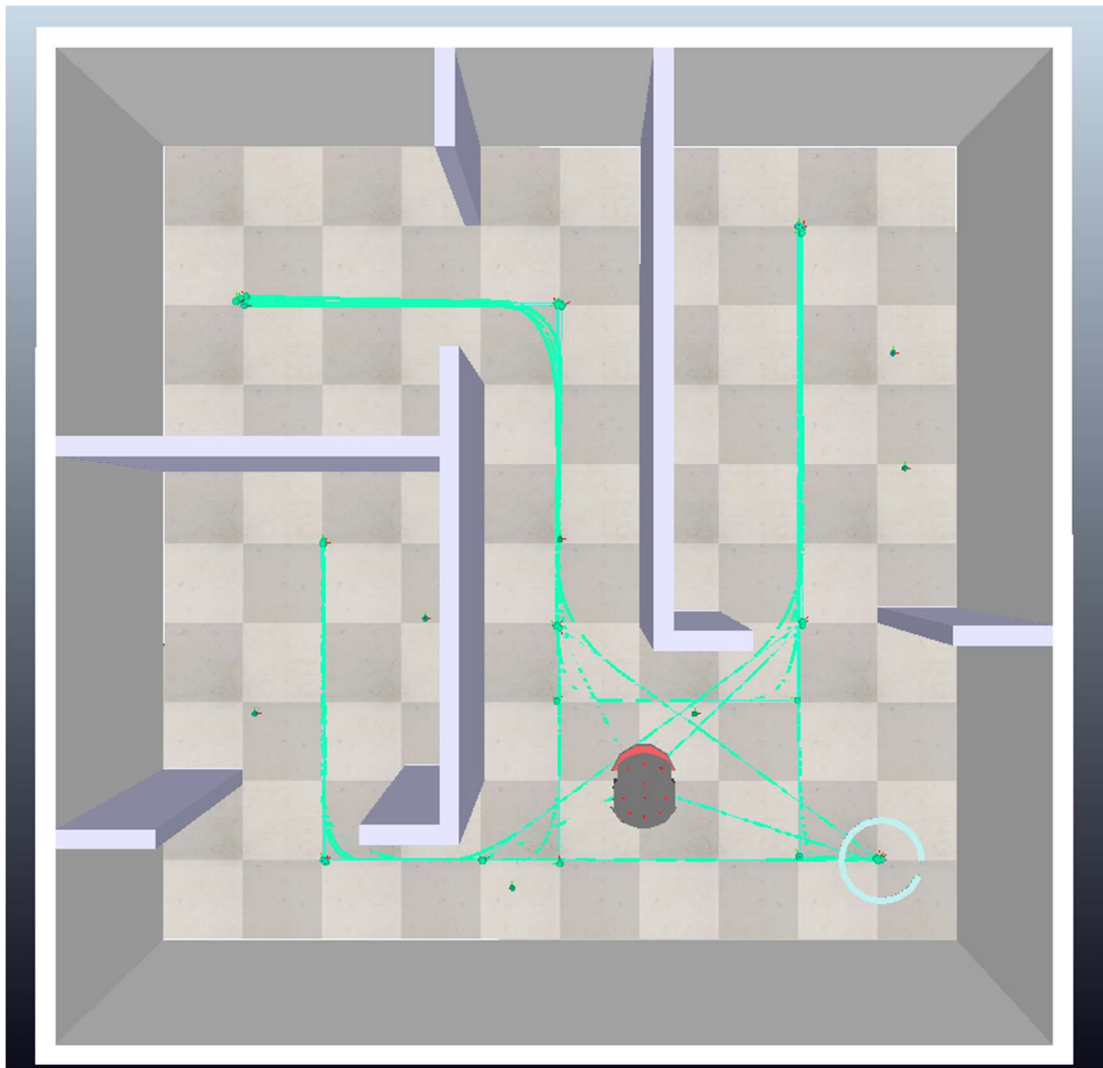


Fig. 2. Entorno en Coppelius

El Pioneer P3DX se desplaza entre nodos mediante movimiento cinemático directo sobre las trayectorias, simplificando el sistema y centrando el problema en la decisión de alto nivel.

3.2.2. Script Lua: control del robot

El script `pioneer.lua` gestiona tres funciones principales:

El sistema de navegación implementa la búsqueda del camino más corto mediante Dijkstra sobre el grafo de rutas, el seguimiento de trayectorias y la realineación del robot al llegar a cada nodo.

El modelo de batería simula un ciclo realista. La descarga se divide en dos fases: meseta inicial al 100% durante un tiempo configurable, seguida de caída lineal hasta el agotamiento. La carga sigue un proceso proporcional al déficit hasta el 100%.

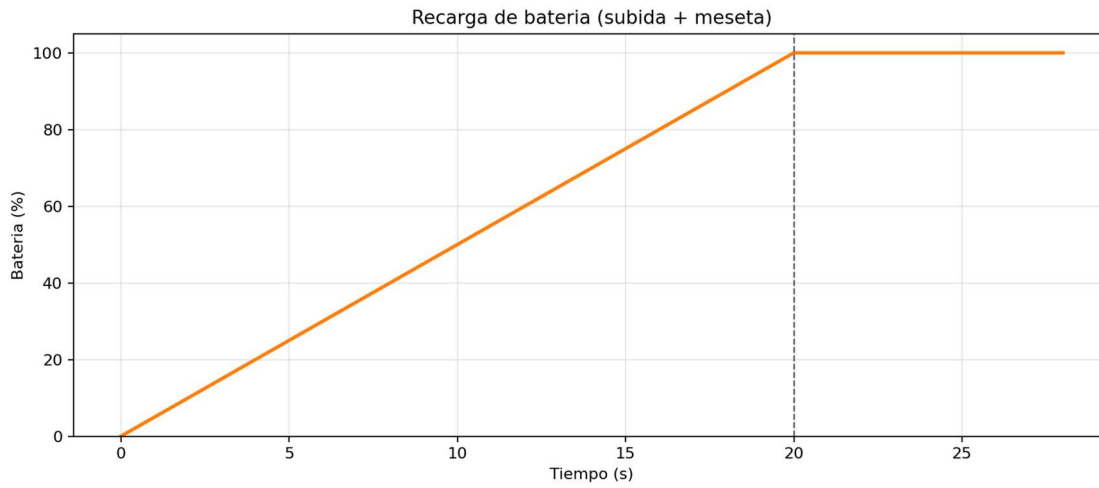


Fig. 3. Modelo recarga batería

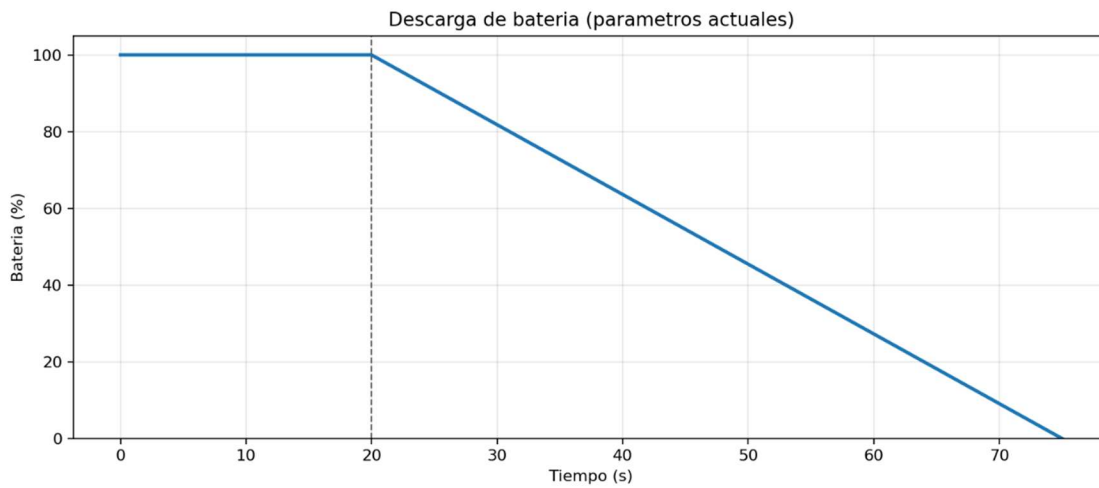


Fig. 4. Modelo de descarga de batería

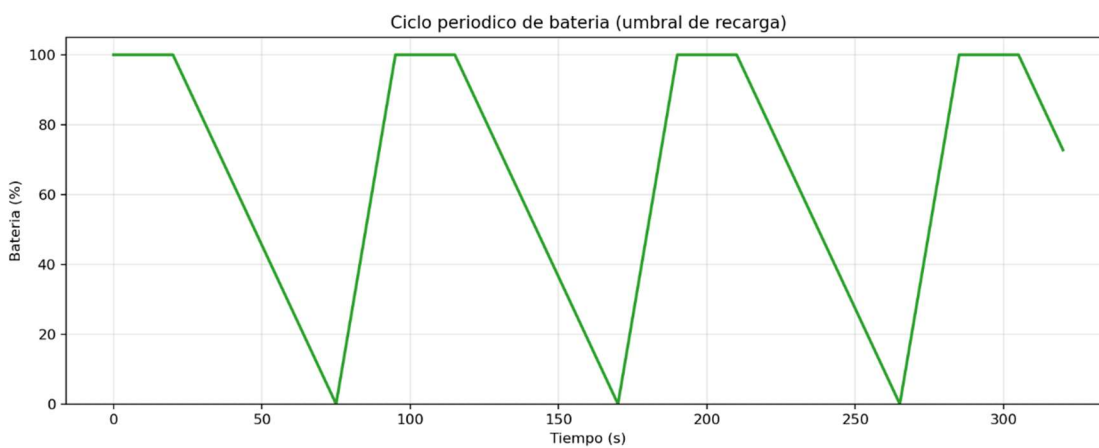


Fig. 5. Ciclo de batería periódico

El sistema de comandos remotos acepta: goTo:<nodo> para desplazarse a un destino; reset para reiniciar el entorno; y stop para detener el robot. El estado se publica continuamente: nivel de batería, nodo actual, estado interno y flag de batería agotada.

3.2.3. Comunicación Python-CoppeliaSim

La comunicación se establece mediante `coppelasim_zmqremoteapi_client`. La clase `RobotCoppeliaSim` encapsula toda la lógica: `go_to(node)` envía el comando y bloquea hasta que el robot reporta `arrived`, `depleted` o `error`; `reset()` reinicia y espera `ready`. Esta sincronización garantiza que cada `step()` de `Gymnasium` corresponde a un desplazamiento completo del robot en la simulación.

3.3. Entorno de aprendizaje por refuerzo

3.3.1. Espacio de observaciones

El vector de observación tiene seis componentes de tipo `float32`:

Índice	Variable	Rango
0	Nodo actual (Hab1=0, Hab2=1, Hab3=2, C=3)	[0, 3]
1	Nivel de batería	[0, 100]
2	Visitas a Hab1 en el episodio	[0, 999]
3	Visitas a Hab2 en el episodio	[0, 999]
4	Visitas a Hab3 en el episodio	[0, 999]
5	Visitas a C en el episodio	[0, 999]

Los contadores de visitas proporcionan al agente información sobre el desequilibrio acumulado entre habitaciones, lo que le permite aprender a priorizar aquellas que han sido menos visitadas. El nivel de batería es fundamental para que el agente aprenda a anticipar la necesidad de recarga.

3.3.2. Espacio de acciones

El espacio de acciones inicial es discreto con cuatro valores posibles:

Acción	Destino
0	Hab1
1	Hab2
2	Hab3
3	C (estación de carga)

Cada acción ordena al robot desplazarse al nodo correspondiente. Si el robot ya se encuentra en ese nodo, el desplazamiento se completa de forma inmediata sin consumo adicional de batería. En lugar de impedir explícitamente seleccionar el nodo actual, se permitió dicha acción con el objetivo de que el propio agente aprendiese, a través de la función de recompensa y de la experiencia acumulada durante el entrenamiento, cuándo repetir una acción carece de utilidad estratégica.

3.3.3. Condiciones de finalización del episodio

Un episodio puede terminar de dos formas. La finalización por `terminated=True` ocurre cuando la batería llega a cero, lo que constituye un fallo que el agente debe aprender a evitar. La finalización por `truncated=True` ocurre cuando se supera el número máximo de pasos por episodio, fijado por defecto en 50. Este límite garantiza que los episodios tengan una duración acotada durante el entrenamiento.

3.3.4. Función de recompensa

El diseño de la función de recompensa es el núcleo experimental de este trabajo y evoluciona a lo largo de los distintos experimentos. La estructura general se mantiene constante: se otorga una recompensa positiva por visitar habitaciones de forma equilibrada, se penaliza en distintos grados la visita innecesaria a la estación de carga, y se aplica una penalización severa si la batería se agota. El capítulo 5 detalla la evolución de esta función a lo largo de los experimentos realizados.

3.4. Proceso de entrenamiento y evaluación

El entrenamiento se ejecuta mediante el script `train.py`, que instancia el entorno `RobotEnv`, verifica su compatibilidad con `Gymnasium` mediante `check_env`, crea un modelo PPO con política `MlpPolicy` y lanza el entrenamiento durante un número configurable de pasos totales. El modelo resultante se guarda en disco en formato `.zip` para su posterior evaluación.

La evaluación se realiza en dos modalidades. La evaluación rápida ejecuta un número reducido de episodios con política determinista (`deterministic=True`) y muestra por consola las métricas principales de cada episodio. La evaluación profunda también utiliza una política determinista ejecuta 500 episodios y calcula un conjunto amplio de métricas estadísticas: recompensa media y por percentiles, tasa de cobertura completa, tiempo proporcional en la estación de carga, batería mínima alcanzada, desbalance entre habitaciones y número de cargas por episodio. Los resultados se exportan en formato CSV y se visualizan mediante histogramas y diagramas de caja.

3.5. Gestión del tiempo en la simulación

El tiempo de simulación es un elemento transversal en este proyecto que afecta a tres aspectos distintos: el coste computacional del entrenamiento, la dinámica de la batería del robot y la información disponible para el agente en su vector de observación. Esta sección describe los distintos conceptos de tiempo presentes en el sistema y las estrategias exploradas para reducir el coste temporal del entrenamiento.

3.5.1. Tipos de tiempo en el sistema

En el sistema desarrollado coexisten tres nociones de tiempo con funciones distintas.

- El tiempo real de pared (`wall time`) es el tiempo físico transcurrido en el reloj del sistema operativo durante el entrenamiento. Determina la duración total de

cada experimento, pero no afecta por sí mismo a la dinámica del robot ni a las señales observadas por el agente.

- El tiempo de simulación es el tiempo interno de CoppeliaSim, que avanza cuando se ejecutan llamadas a `sim.step()`. Cada paso de simulación incrementa el reloj interno del simulador y proporciona la referencia temporal sobre la que se definen los desplazamientos, las transiciones de estado y el modelo de batería.
- El tiempo virtual (`virtualTime`) es una variable interna del script Lua que representa el tiempo lógico utilizado por el entorno para calcular la duración de las acciones y la evolución de la batería. Su valor se actualiza proporcionalmente al tiempo de simulación mediante un factor configurable denominado `SIM_ACCELERATION`. Todos los cálculos dependientes del tiempo, incluida la duración de las trayectorias, el tiempo asociado a la acción stop y la señal `robot_last_action_duration`, se expresan en esta escala virtual. De este modo, al modificar `SIM_ACCELERATION` se reduce o aumenta el tiempo de pared necesario para entrenar, pero se mantiene invariante la dinámica que percibe el agente.

3.5.2. Aceleración de la simulación

Dado el alto coste temporal del entrenamiento con CoppeliaSim, se exploró la posibilidad de reducir el tiempo de pared sin modificar la lógica del entorno. La estrategia adoptada consiste en aumentar el valor de `SIM_ACCELERATION` por encima de 1.0, de forma que cada paso de simulación represente más tiempo virtual sin incrementar el número de llamadas a `sim.step()`.

Se realizaron pruebas con distintos valores del factor de aceleración. A partir de valores superiores a 3.0 se observaron pérdidas de estabilidad en la simulación, con pequeñas imprecisiones acumuladas en el movimiento del robot. Con `SIM_ACCELERATION=3.0` el comportamiento se mantuvo estable y las métricas de aprendizaje fueron equivalentes a las obtenidas sin aceleración, tal como se verifica en el experimento `exp_015`. Este valor se adoptó como configuración estándar a partir de la línea de experimentos 4, reduciendo el tiempo de pared de cada entrenamiento de forma significativa sin alterar el comportamiento funcional del entorno.

3.5.3. Modo teleport

Como alternativa a la aceleración por escalado temporal, se exploró un modo de movimiento alternante denominado teleport. En este modo, en lugar de seguir la trayectoria física entre nodos paso a paso, el robot se teletransporta directamente al nodo destino y el tiempo virtual avanza en el mismo instante en la cantidad que habría consumido el desplazamiento real, calculada a partir de la longitud del camino y la velocidad configurada.

El modo teleport elimina por completo el tiempo de pared asociado al movimiento, ya que no requiere iteraciones de `sim.step()` para la navegación. Sin embargo, su implementación presenta dificultades en la gestión de la recarga: cuando el robot se

teletransporta a C, el sistema de detección de proximidad a la estación de carga no se activa de forma fiable, lo que puede provocar que la batería no se recargue correctamente. Por este motivo, el modo teleport no se utilizó en los experimentos de este trabajo y el modo path con SIM_ACCELERATION=3.0 se mantuvo como configuración de referencia.

4

Metodología

Este capítulo describe el proceso de trabajo seguido durante el desarrollo del TFG. Dado que el núcleo del proyecto es el diseño de la función de recompensa para el agente de aprendizaje por refuerzo, la metodología adoptada es de naturaleza iterativa y experimental: cada experimento parte de los resultados del anterior, introduce una modificación concreta y analiza su efecto sobre el comportamiento del agente.

4.1. Enfoque iterativo

El desarrollo del sistema no siguió un proceso lineal de diseño, implementación y evaluación, sino un ciclo continuo en el que cada iteración comprende las siguientes fases:

En primer lugar, se define una hipótesis sobre el comportamiento esperado del agente dado el diseño actual de la función de recompensa. Esta hipótesis puede surgir de la observación de resultados previos, del análisis de las métricas de evaluación o de la detección de comportamientos no deseados en el agente entrenado.

A continuación, se implementa la modificación correspondiente en `robot_env.py`, ajustando la función de recompensa de forma controlada y manteniendo el resto del sistema invariante. Cada variante se documenta en su carpeta de experimento con una descripción del cambio introducido y el resultado esperado.

Posteriormente, se lanza el entrenamiento del agente durante un número fijo de pasos mediante `train.py`, se monitorizan las curvas de aprendizaje en TensorBoard y se verifica que el entrenamiento ha convergido antes de evaluar.

Finalmente, se ejecuta la evaluación profunda con 500 episodios mediante `evaluate_deep.py` y se analizan las métricas obtenidas frente a los criterios de éxito definidos y frente a los resultados de los experimentos anteriores.

4.2. Criterios de evaluación

Para poder comparar experimentos de forma objetiva, se definió un conjunto de criterios de éxito cuantitativos organizados en tres bloques.

El bloque A recoge los requisitos obligatorios: una tasa de cobertura completa del 100%, es decir, el agente debe visitar las tres habitaciones en todos los episodios evaluados, y cero episodios con batería agotada en los 500 episodios de evaluación profunda. Ningún experimento se considera satisfactorio si no cumple estos dos requisitos.

El bloque B agrupa las métricas de eficiencia y estabilidad: tiempo medio en la estación de carga inferior al 28%, número medio de cargas por episodio inferior a 12.5, paso medio de primera cobertura completa inferior a 6, y recompensa por paso media superior a 0.37 con baja dispersión.

El bloque C recoge las métricas de balance energético y equilibrio de patrulla: desviación típica media de visitas entre habitaciones inferior a 1.30, rango máximo de visitas inferior a 3, y batería mínima alcanzada superior al 30% en el percentil 10.

Este conjunto de criterios permite distinguir con precisión entre experimentos que resuelven el problema correctamente y aquellos que lo hacen de forma eficiente y estable, lo cual es relevante para orientar las decisiones de diseño en cada iteración.

4.3. Estructura de experimentos

Cada experimento se almacena en una carpeta independiente bajo el directorio `experiments/`, con una nomenclatura descriptiva que refleja la variante de recompensa implementada. Dentro de cada carpeta se encuentran el modelo entrenado en formato `.zip`, los logs de TensorBoard, el fichero `descripcion.txt` con la especificación de la función de recompensa y los resultados de la evaluación profunda en formato CSV y gráficas generadas.

Esta organización facilita la reproducibilidad de los experimentos y la comparación directa entre variantes, ya que todos comparten la misma infraestructura de simulación, los mismos hiperparámetros de PPO y el mismo número de pasos de entrenamiento, variando únicamente la función de recompensa definida en `robot_env.py`.

4.4. Herramientas de monitorización y análisis

Durante el entrenamiento se utiliza TensorBoard para visualizar en tiempo real la evolución del aprendizaje. Las métricas que resultan más informativas en el contexto de este trabajo son la recompensa episódica media (*rollout/ep_rew_mean*), que permite comprobar si el agente está mejorando, y la longitud media de los episodios (*rollout/ep_len_mean*), cuya evolución a menudo revela comportamientos inesperados antes de que la evaluación profunda los confirme.

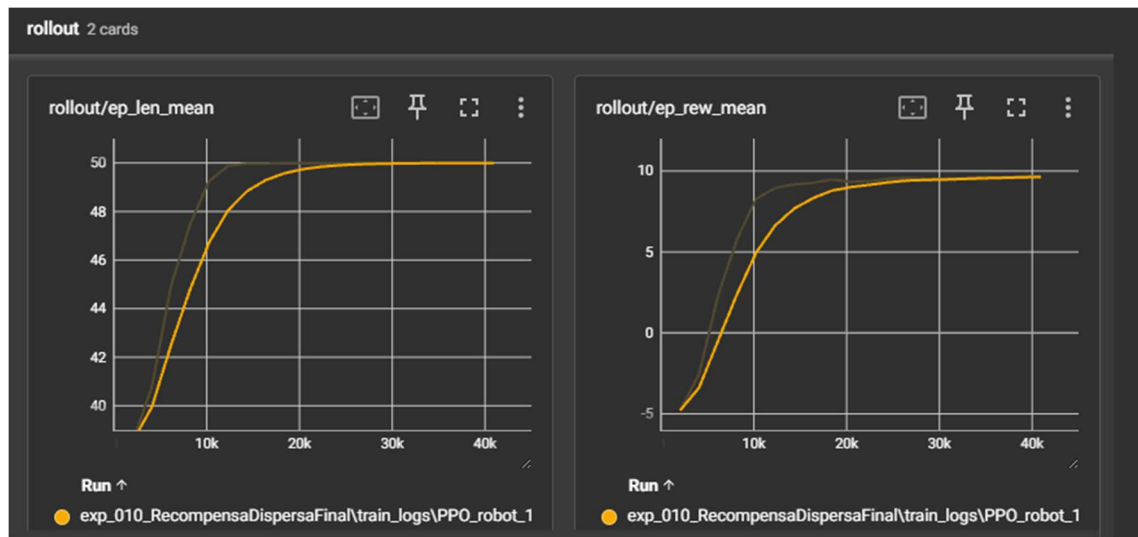


Fig. 6. Ejemplo de curva de Tensorflow

Además de las métricas de entrenamiento, TensorBoard expone las métricas internas de PPO: la pérdida del crítico (*train/value_loss*), la pérdida de la política (*train/policy_gradient_loss*) y la entropía de la política (*train/entropy_loss*). Una entropía que cae muy rápido indica que el agente converge a una estrategia muy determinista desde el principio, posiblemente sin explorar suficiente. Una pérdida del crítico que no converge sugiere que la función de valor no está aprendiendo a estimar correctamente el retorno, lo que puede indicar un problema en el diseño de la recompensa. Estas señales orientaron varias decisiones de rediseño a lo largo del proyecto.

Para el análisis posterior al entrenamiento se emplean pandas y matplotlib, que procesan los ficheros CSV generados por `evaluate_deep.py` y producen histogramas, diagramas de caja y gráficas de distribución de las métricas clave. Estos resultados se utilizan tanto para la toma de decisiones en la siguiente iteración como para la elaboración de las figuras incluidas en el capítulo 5.

5

Experimentos y resultados

Este capítulo describe el proceso iterativo de diseño del entorno de aprendizaje y los resultados obtenidos en cada experimento. Los dieciocho experimentos realizados se organizan en cuatro grandes líneas de evolución, centradas respectivamente en el diseño de la función de recompensa, el uso de recompensas dispersas, la ampliación del espacio de acciones y la incorporación de información temporal en la observación del agente.

- La primera línea, formada por los experimentos exp_001 a exp_008, explora distintas variantes de recompensa densa basada en el equilibrio de visitas entre habitaciones y diferentes estrategias de penalización por visitar la estación de carga.
- La segunda línea, exp_009 a exp_012, estudia la transición hacia una función de recompensa dispersa calculada al final del episodio.
- La tercera línea, exp_013 y exp_014, amplía el espacio de acciones incorporando una acción de parada controlada y analiza su efecto sobre el comportamiento del agente.
- La cuarta línea, exp_015 a exp_018, analiza el efecto de introducir información temporal explícita en la observación del agente.

Cada sección describe la hipótesis que motivó el experimento, el diseño implementado y el comportamiento observado durante la evaluación. El capítulo concluye con una comparativa global de resultados entre todos los experimentos realizados.

5.1. Recompensa base: visitas equilibradas a habitaciones

5.1.1. exp_001 y exp_001_remake: recompensa por media de habitaciones

El primer experimento establece el punto de partida del sistema y actúa como configuración base del problema de patrullaje. La hipótesis inicial es que recompensar al agente únicamente por visitar habitaciones que estén por debajo de la media de visitas del episodio puede ser suficiente para inducir un comportamiento de exploración equilibrada, siempre que la penalización por agotamiento de batería sea lo suficientemente severa como para forzar visitas a la estación de carga.

La función de recompensa es la siguiente: se otorga +1.0 por visitar Hab1, Hab2 o Hab3 cuando el número de visitas previas a esa habitación es inferior a la media de visitas de las tres habitaciones; ir a C no genera ni recompensa ni penalización; y agotar la batería genera una penalización de -10.0 con terminación del episodio. Este diseño sirve como formulación inicial del problema de patrullaje con incentivo local basado en el desequilibrio de visitas.

La variante exp_001_MediaSoloHabsRemakeobs introduce un cambio en la representación del espacio de observaciones, que pasa a estar compuesto por seis componentes al incorporar los contadores batería y nodo actual. De este modo, el agente cuenta con una representación más detallada del estado del entorno, en particular respecto al desequilibrio acumulado entre habitaciones.

Los resultados de la evaluación profunda revelan un comportamiento degenerado, característico de esta configuración inicial: el agente aprende a ir directamente a C y permanecer allí durante todo el episodio, obteniendo recompensa 0 pero evitando el riesgo de penalización por batería agotada. La tasa de cobertura completa es del 0% y el tiempo en C es del 100%. Este comportamiento se explica por la ausencia de penalización por visitar C, lo que convierte esta acción en una estrategia segura desde el punto de vista de la política aprendida, al eliminar completamente el riesgo de penalización de -10.0 sin coste asociado.

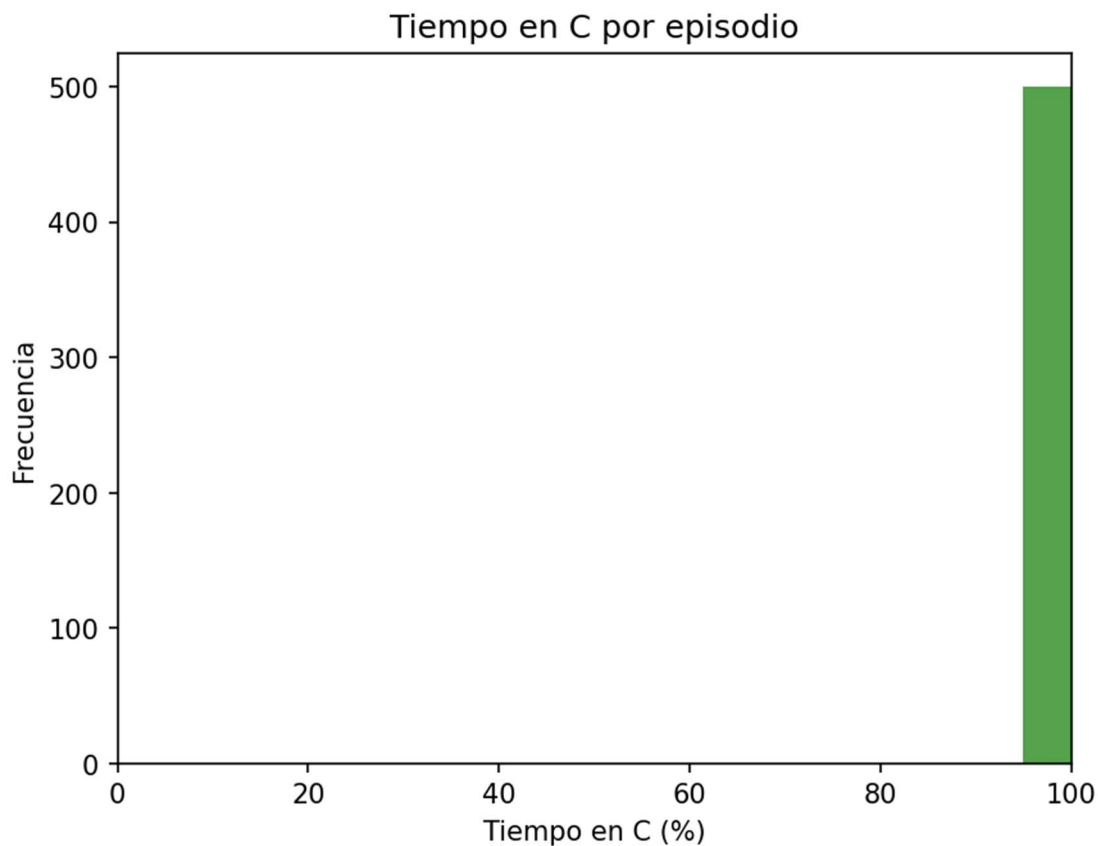


Fig. 7. Tiempo en C exp_001

5.2. Incorporación de C en la media de visitas

5.2.1. exp_002: C en la media, sin recompensa por C

La segunda línea de experimentación introduce una modificación sobre el comportamiento observado en el experimento anterior, incorporando la estación de carga C en el cálculo de la media de visitas. La hipótesis es que, al incluir C en la referencia estadística, las visitas a esta estación incrementarían el umbral medio del episodio, reduciendo así el incentivo del agente a permanecer en ella de forma continuada.

La función de recompensa mantiene el mismo esquema de exp_001, con la diferencia de que la media de visitas se calcula ahora incluyendo el contador de C junto con los de Hab1, Hab2 y Hab3. No obstante, visitar C sigue sin generar recompensa directa.

Los resultados muestran un cambio en el régimen de comportamiento del agente respecto al caso anterior. Aunque la cobertura completa continúa siendo del 0%, se observa una reducción del tiempo en C, que desciende al 67.9%. Sin embargo, este cambio no se traduce en una mejora del comportamiento de patrullaje, ya que el agente desarrolla una política altamente sesgada, concentrando la mayoría de las visitas en Hab3 y mostrando nula exploración de Hab1.

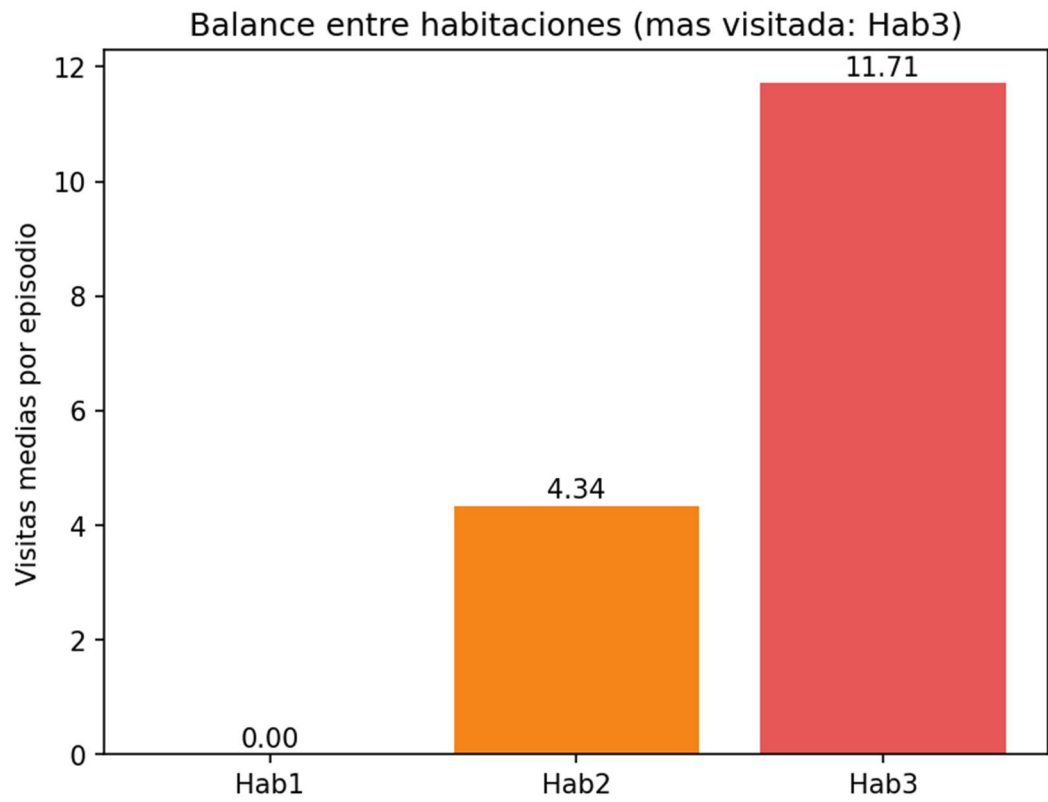


Fig. 8. Balance entre habitaciones exp002

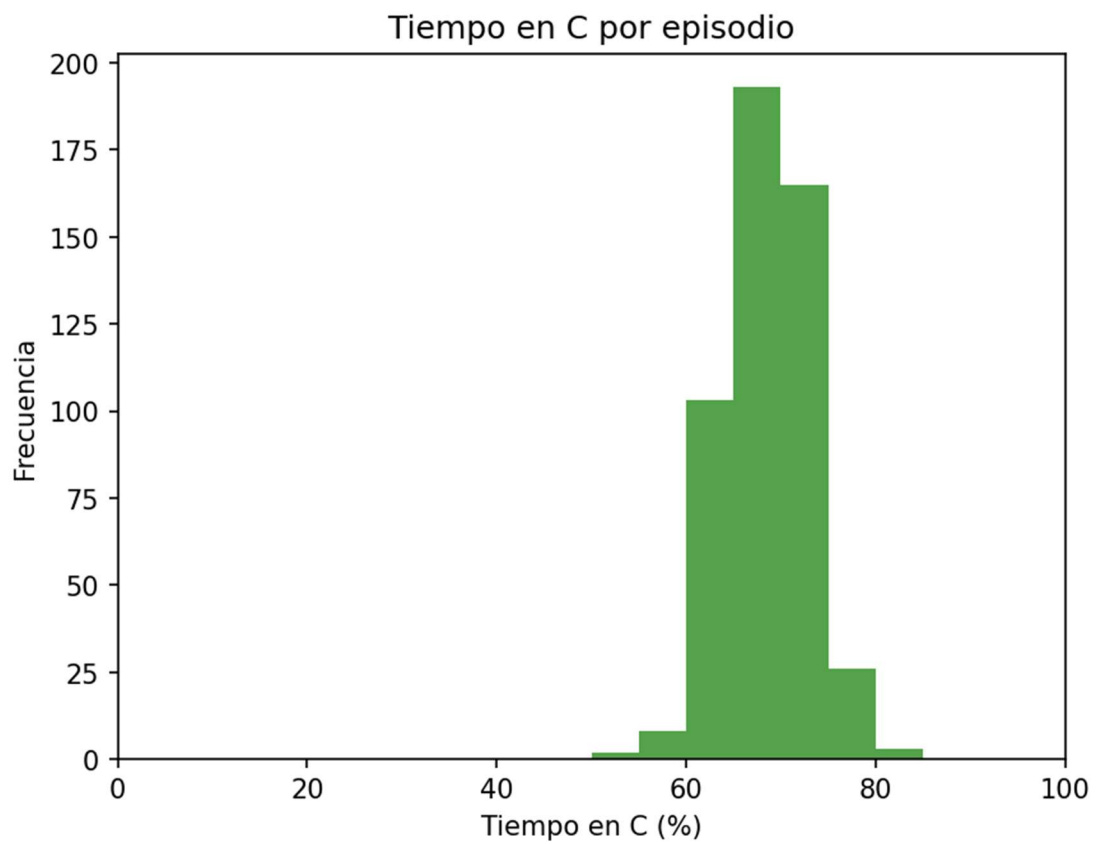


Fig. 9. Tiempo en C exp_002

En conjunto, el experimento muestra que la inclusión de C en la media altera el comportamiento observado en exp_001, reduciendo el tiempo que el agente permanece en la estación de carga. Sin embargo, esta modificación no resulta suficiente para producir una política de patrullaje satisfactoria. Aunque el agente abandona la estrategia de permanecer permanentemente en C, sigue sin alcanzar cobertura completa y desarrolla un comportamiento sesgado hacia un subconjunto de habitaciones.

5.2.2. exp_003: C en la media, con recompensa por C

El tercer experimento extiende el anterior otorgando recompensa +1.0 también por visitar C cuando está por debajo de la media. La hipótesis es que tratar C exactamente igual que las habitaciones podría inducir un comportamiento más equilibrado al unificar el criterio de recompensa.

Los resultados muestran un cambio importante respecto a exp_002. La cobertura completa alcanza el 100%, lo que indica que el agente aprende a visitar todas las habitaciones del entorno. Sin embargo, esta mejora viene acompañada de un comportamiento no deseado: el agente descubre que la estación de carga puede utilizarse como una fuente adicional de recompensa sin apenas riesgo asociado.

El tiempo en C alcanza el 48.4% y el número medio de cargas por episodio asciende a 16.25. Además, la batería mínima media es del 99.1%, lo que indica que el agente recarga de forma prácticamente continua y rara vez permite que la batería descienda de forma significativa. En lugar de utilizar C únicamente cuando resulta necesario desde el punto de vista energético, la política aprendida incorpora visitas frecuentes a la estación de carga como mecanismo para maximizar la recompensa.

En conjunto, el experimento muestra que incluir C dentro del mismo esquema de recompensa que las habitaciones favorece la cobertura del entorno, pero introduce un incentivo no deseado hacia el uso excesivo de la estación de carga. La formulación consigue eliminar el problema de cobertura observado en exp_001 y exp_002, pero genera una política energéticamente poco realista basada en recargas continuas.

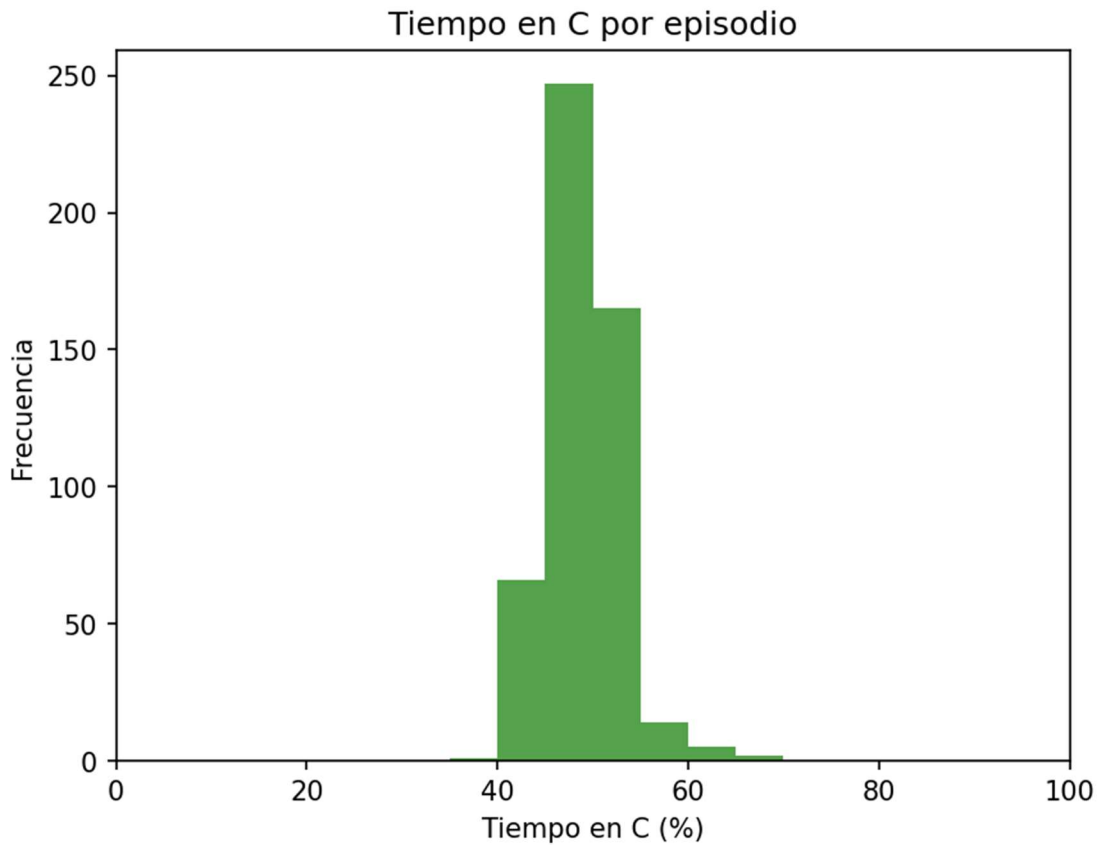


Fig. 10. Tiempo en Carga exp_003

5.3. Penalización por visitar C

5.3.1. exp_004: penalización fija de -0.1 por ir a C

La cuarta línea introduce por primera vez una penalización explícita por visitar C, con el objetivo de desincentivar las cargas innecesarias. La función de recompensa recupera el esquema de exp_001 (media solo de habitaciones, sin C) y añade una penalización fija de -0.1 cada vez que el agente elige C como destino.

Los resultados muestran que la cobertura completa se mantiene en el 100% y que la batería no llega a agotarse en ningún episodio, lo que satisface el bloque A de los criterios de éxito. Sin embargo, el comportamiento del agente sigue estando lejos del objetivo de eficiencia: el tiempo en C se sitúa en el 51.2% y el número medio de cargas asciende a 18.03, valores que indican un uso excesivo de la estación de carga.

La batería media al recargar es del 100%, lo que confirma que el agente sigue realizando las recargas en estados de batería completamente llena o prácticamente llena. En este contexto, la penalización de -0.1 resulta insuficiente para alterar la política aprendida, ya que no compensa el incentivo estructural de recurrir a C como acción segura dentro del horizonte del episodio.

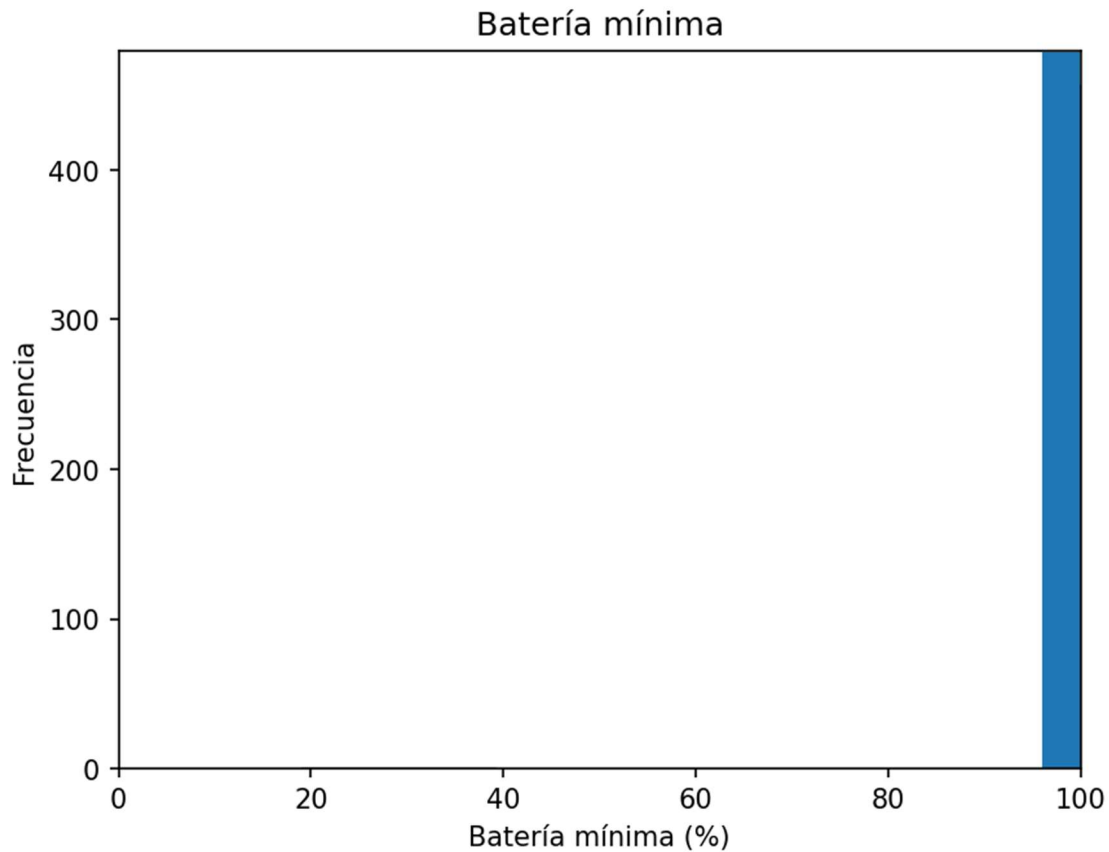


Fig. 11. Batería mínima exp_004

5.4. Penalización adaptativa y refinación del balance

5.4.1. exp_005: penalización adaptativa según nivel de batería

El quinto experimento introduce una penalización adaptativa por ir a C en función del nivel de batería en el momento de la visita. La hipótesis es que penalizar más severamente las visitas a C cuando la batería está alta y menos cuando está baja permitirá al agente aprender que recargar con batería alta es ineficiente, mientras que recargar con batería baja es casi obligatorio.

La estructura de penalización es la siguiente: batería superior al 70%, penalización de -0.4; batería entre el 40% y el 70%, penalización de -0.2; batería igual o inferior al 40%, penalización de -0.05. El resto de la función de recompensa se mantiene igual que en exp_004.

Los resultados muestran una mejora respecto a las configuraciones anteriores en términos de eficiencia global. La cobertura completa se mantiene en el 100% sin agotamientos de batería, satisfaciendo el bloque A. El tiempo en C desciende al 29.7% y las cargas medias se reducen a 10.10, mientras que la recompensa por paso media aumenta a 0.365.

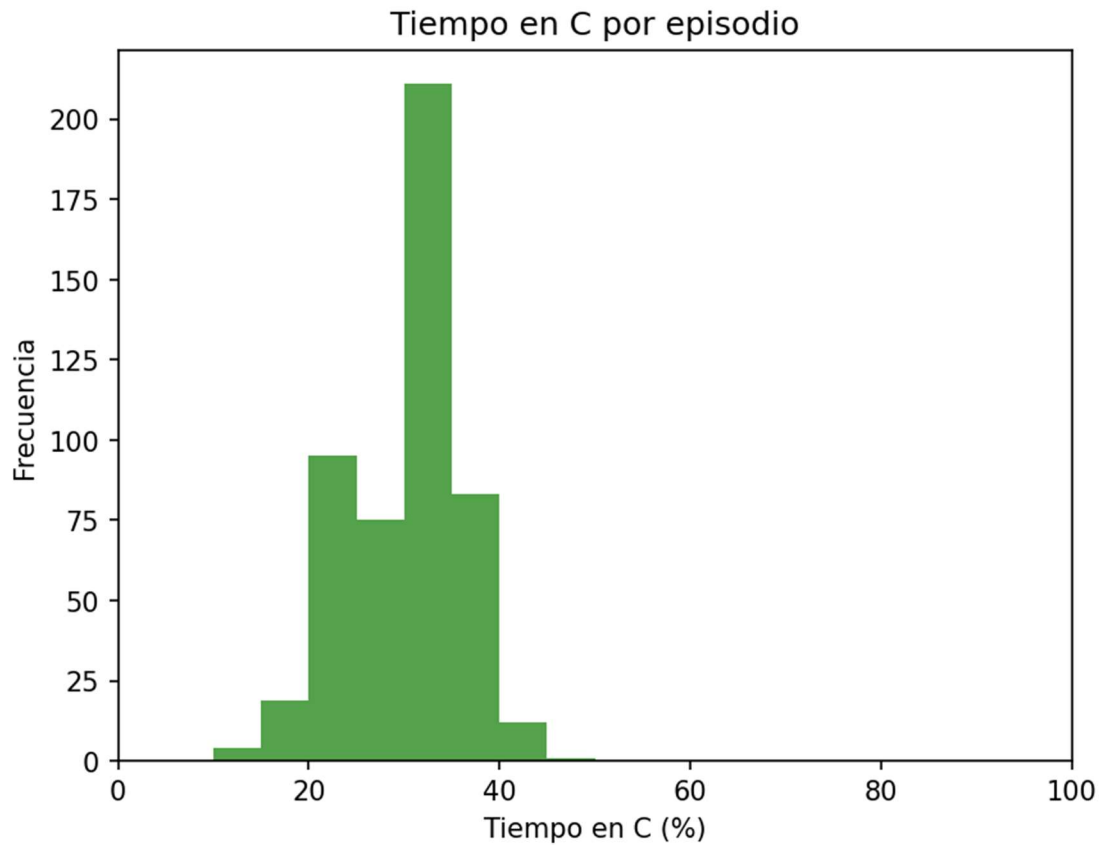


Fig. 12. Tiempo en C exp_005

En cuanto a la gestión energética, la batería media al recargar se sitúa en el 87.3%, lo que indica una cierta reducción en la tendencia a recargar con batería máxima, aunque el comportamiento sigue siendo mayoritariamente conservador. El percentil 10 de batería mínima (27%) sugiere que en algunos episodios el agente sí alcanza niveles de batería más ajustados antes de recargar, aunque no de forma consistente.

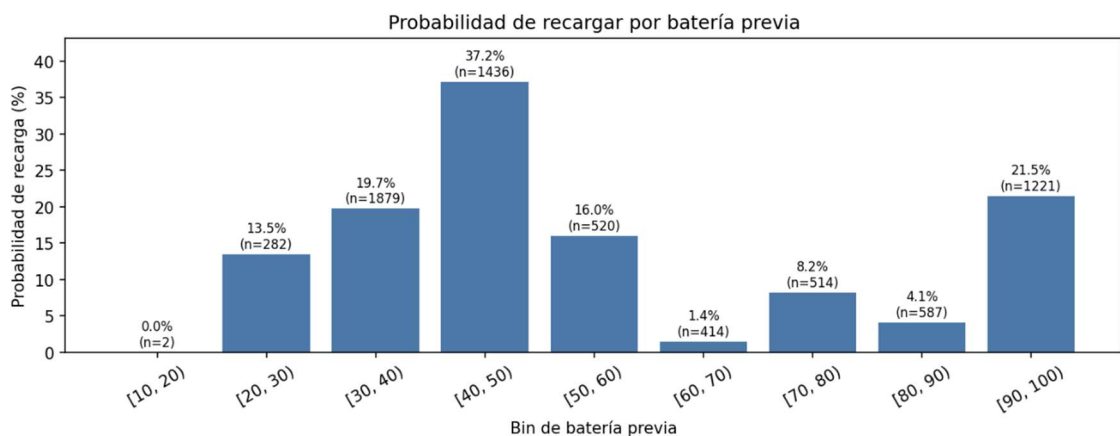


Fig. 13. Probabilidad de recarga por batería previa exp_005

El desbalance entre habitaciones presenta una desviación típica media de 1.61 y un rango medio de 3.75, indicando una mejora moderada en la uniformidad de visitas respecto a experimentos anteriores, aunque sin alcanzar aún un equilibrio estable.

5.4.2. exp_006: penalización por sobrerrepresentación de habitación

El sexto experimento parte de exp_005 e introduce una penalización adicional de -0.2 cuando el agente visita una habitación cuyo contador de visitas supera en más de una unidad la media de las tres habitaciones. El objetivo es corregir el sesgo sistemático observado previamente, donde el agente tendía a concentrar visitas en Hab3, generando una distribución desequilibrada.

Los resultados confirman una mejora clara en el objetivo de balance entre habitaciones. El desbalance medio desciende a 1.29 de desviación típica y 2.98 de rango, cumpliendo los objetivos del bloque C. Este cambio indica que la penalización adicional es efectiva para evitar la acumulación excesiva de visitas en una única habitación.

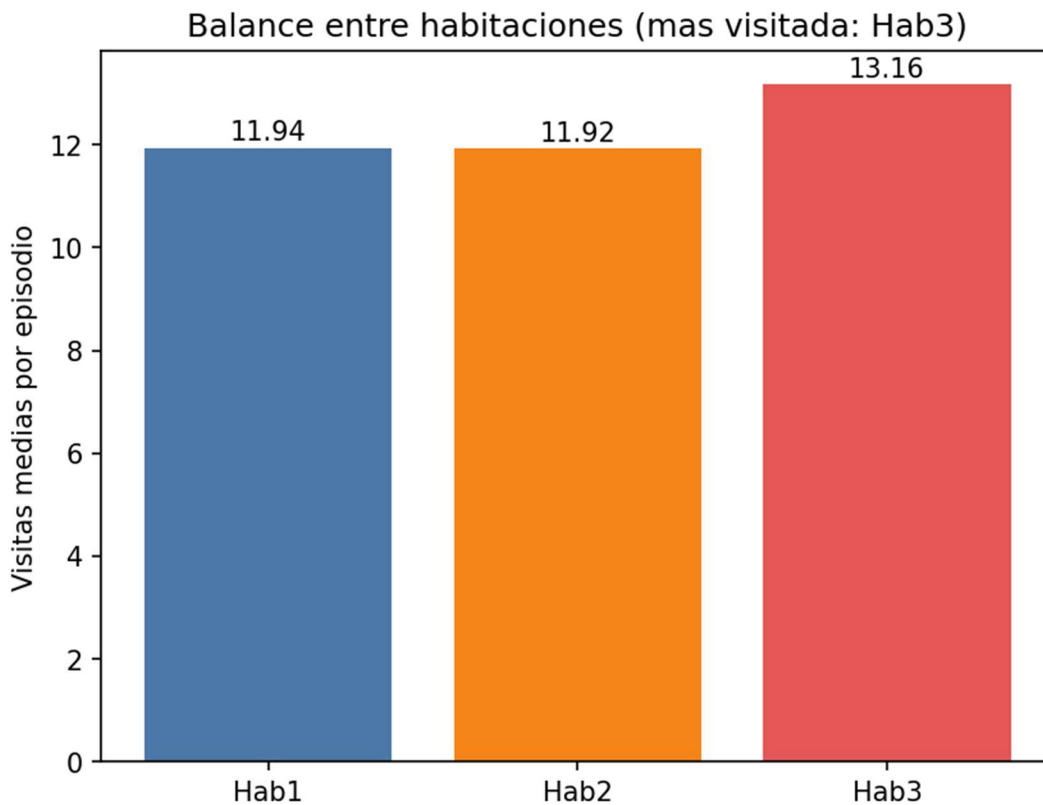


Fig. 14. Balance entre habitaciones exp_006

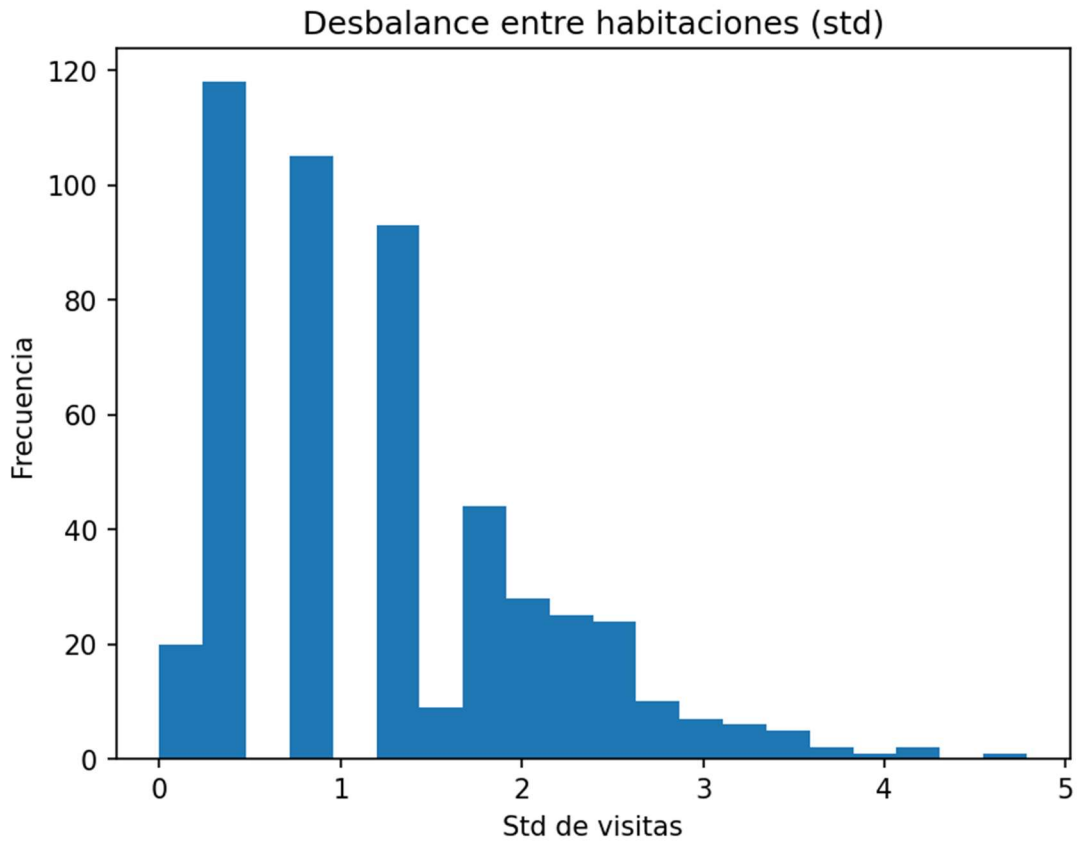


Fig. 15. Desbalance entre habitaciones exp_006

En paralelo, se observa un efecto secundario relevante en la dinámica del episodio: el tiempo en C se reduce al 26.0% y el agente alcanza la cobertura completa en un número muy reducido de pasos (3.7 de media hasta completar la exploración). Esto sugiere una política más agresiva de exploración inicial, con visitas rápidas a las tres habitaciones antes de estabilizar el comportamiento.

Sin embargo, la mejora en el balance no se traduce en una gestión energética más realista. La batería media al recargar permanece alta (98.1%) y el percentil 10 de batería mínima es del 39%, lo que indica que el agente sigue adoptando una estrategia conservadora de recarga temprana, sin ajustar de forma fina el uso de la batería.

5.4.3. exp_007: endurecimiento del umbral de penalización adaptativa

El séptimo experimento endurece la penalización asociada a las visitas a C en condiciones de batería alta, elevando el umbral de penalización fuerte al 85% y aumentando la penalización a -0.6 en ese rango. El objetivo es reducir aún más la frecuencia de recarga prematura observada en los experimentos anteriores, incentivando al agente a retrasar la visita a la estación de carga sin comprometer la seguridad energética.

Los resultados muestran una mejora en las métricas de eficiencia superficial del comportamiento. El tiempo en C desciende al 27.0%, las cargas medias se reducen a 11.00 y el número de pasos necesarios para completar la cobertura se mantiene estable

en 3.7. El balance entre habitaciones también se mantiene controlado, con una desviación típica de 1.30 y un rango de 3.00.

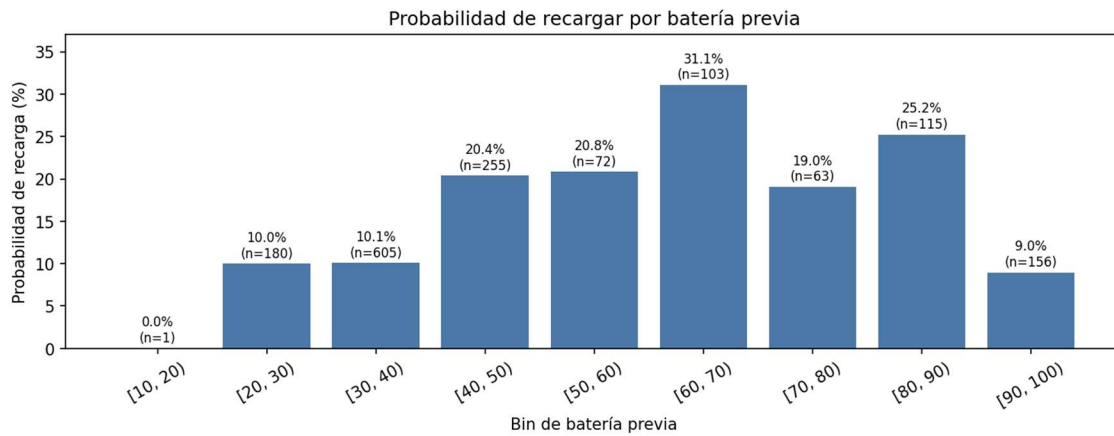


Fig. 16. Probabilidad de recarga por batería previa exp_007

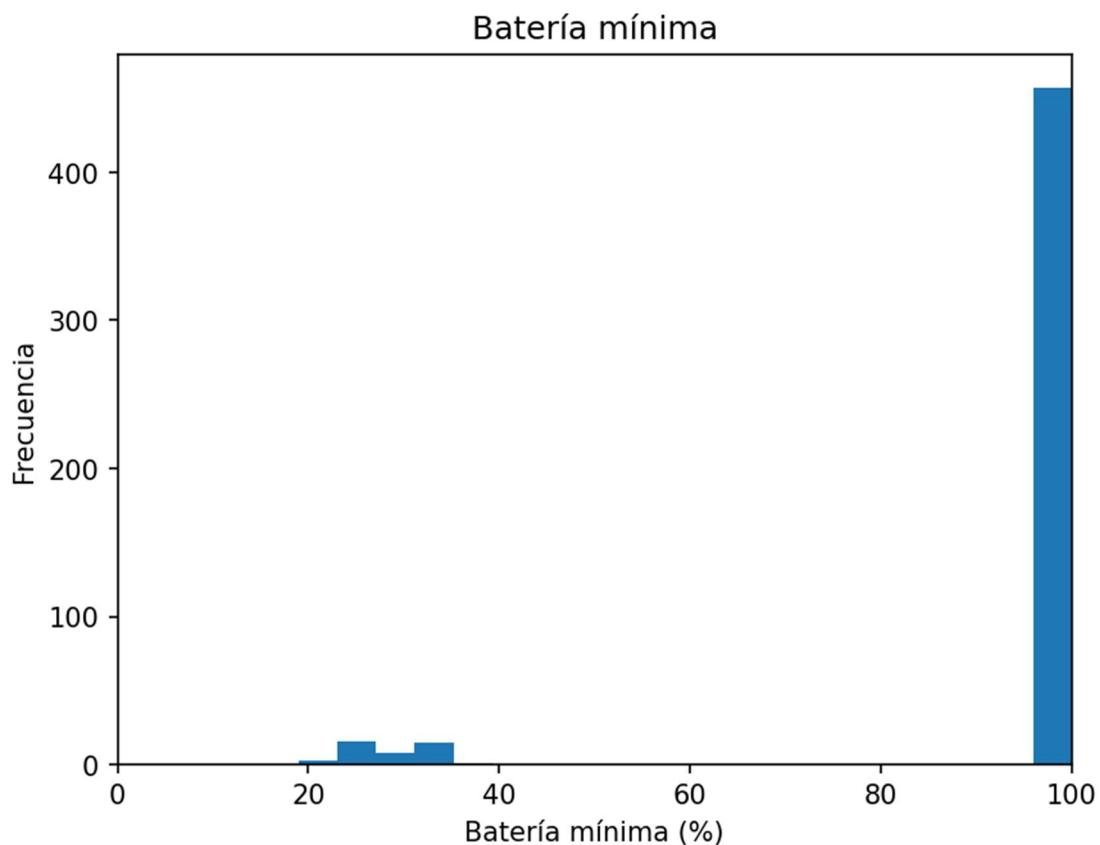


Fig. 17. Batería mínima por episodio exp_007

Sin embargo, el análisis de la dinámica energética revela un cambio cualitativo importante. La batería media al recargar se sitúa en el 98.0% y el percentil 10 de batería mínima también alcanza valores cercanos al 98%, lo que indica que el agente ha aprendido a evitar situaciones de descarga significativa y recargar únicamente cuando la batería se encuentra prácticamente llena.

Este comportamiento sugiere que el endurecimiento excesivo de la penalización no induce una gestión energética más eficiente, sino una reconfiguración del umbral de decisión hacia un extremo conservador. En lugar de fomentar una estrategia de uso progresivo de la batería, el agente adopta una política de recarga anticipada sistemática.

5.4.4. exp_008: penalización fija revisada

El octavo experimento elimina la penalización adaptativa y sustituye todas las variantes por una penalización fija de -0.2 por visitar C, independientemente del nivel de batería. La hipótesis es verificar si el agente puede aprender a gestionar la energía de forma adecuada sin información explícita sobre el nivel de batería en la señal de recompensa, confiando únicamente en la observación del nivel de batería para decidir cuándo recargar.

Los resultados muestran que la cobertura completa se mantiene al 100% con cero agotamientos, pero el tiempo en C sube al 38.4% y las cargas medias aumentan a 14.50 respecto a los experimentos con penalización adaptativa. La recompensa por paso media es de 0.333 y el balance de habitaciones empeora ligeramente con una desviación típica de 1.84. La batería media al recargar es del 99.9%, confirmando que sin información adaptativa en la recompensa el agente no aprende a distinguir cuándo es apropiado recargar.

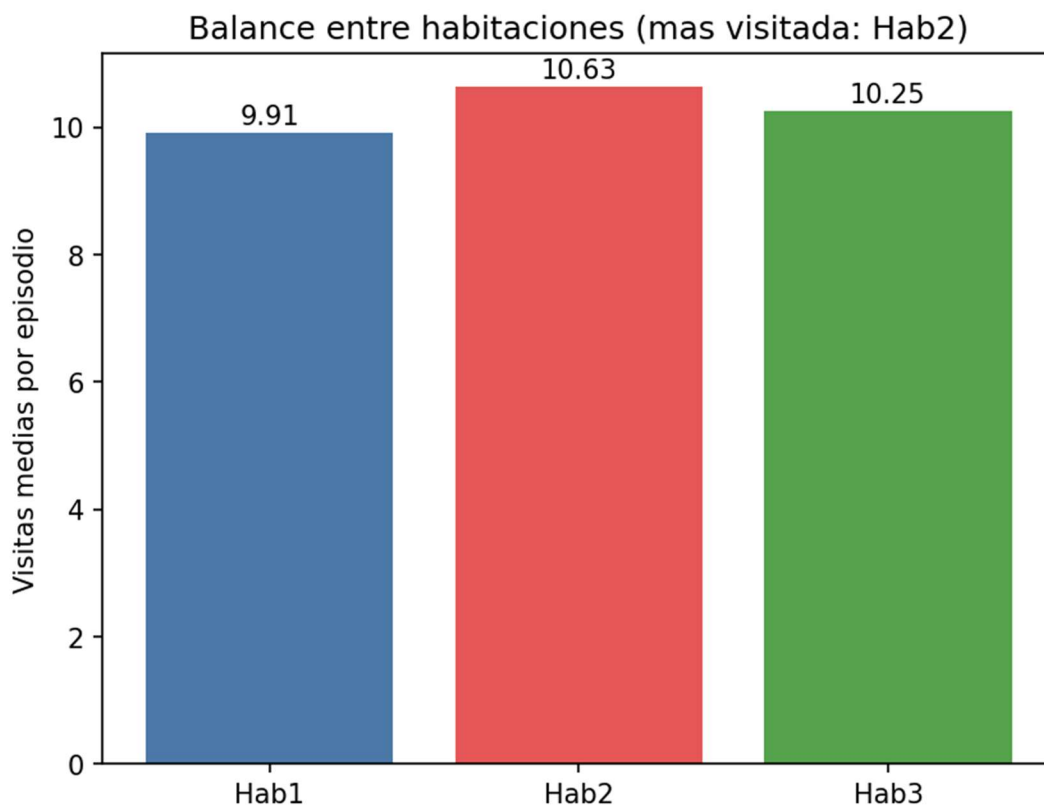


Fig. 18. Balance entre habitaciones exp_008

En cuanto a la dinámica energética, la batería media al recargar alcanza el 99.9%, confirmando que el agente mantiene un patrón de recarga sistemático sin discriminar adecuadamente el momento óptimo para hacerlo. Esto sugiere que la eliminación de la

señal adaptativa reduce la capacidad del agente para ajustar el uso de la batería de forma eficiente.

5.5. Recompensa basada en balance uniforme

Los experimentos de esta línea exploran alternativas a la regla discreta de media de visitas utilizada hasta exp_008, sustituyéndola por una métrica continua de desbalance respecto a la distribución uniforme de visitas entre habitaciones.

5.5.1. exp_009: balance uniforme con penalización densa por C

El experimento exp_009 introduce un cambio sustancial en la formulación de la recompensa, sustituyendo la regla discreta basada en medias por una métrica continua de desbalance respecto a una distribución uniforme de visitas entre habitaciones. El objetivo es analizar si una señal de recompensa densa puede proporcionar un gradiente de aprendizaje más estable que la formulación anterior.

La recompensa asociada a las habitaciones se define a partir de la desviación de la distribución empírica de visitas respecto a la distribución uniforme ideal. Para cada habitación i se calcula su proporción de visitas p_i , y el desbalance global se obtiene como la suma de las desviaciones cuadráticas respecto a $1/3$. A partir de esta medida se define una recompensa escalada en el rango positivo, de forma que valores más cercanos a la distribución uniforme generan mayor recompensa.

La penalización por visitar la estación de carga C se define de forma proporcional a su frecuencia relativa en el episodio.

Esta formulación introduce una característica relevante en la dinámica del aprendizaje: la recompensa se acumula de forma densa a lo largo del episodio, lo que induce una señal no estacionaria cuyo valor crece progresivamente con el número de pasos. Este efecto modifica significativamente la escala de la señal de retorno y dificulta la estabilización del comportamiento del agente.

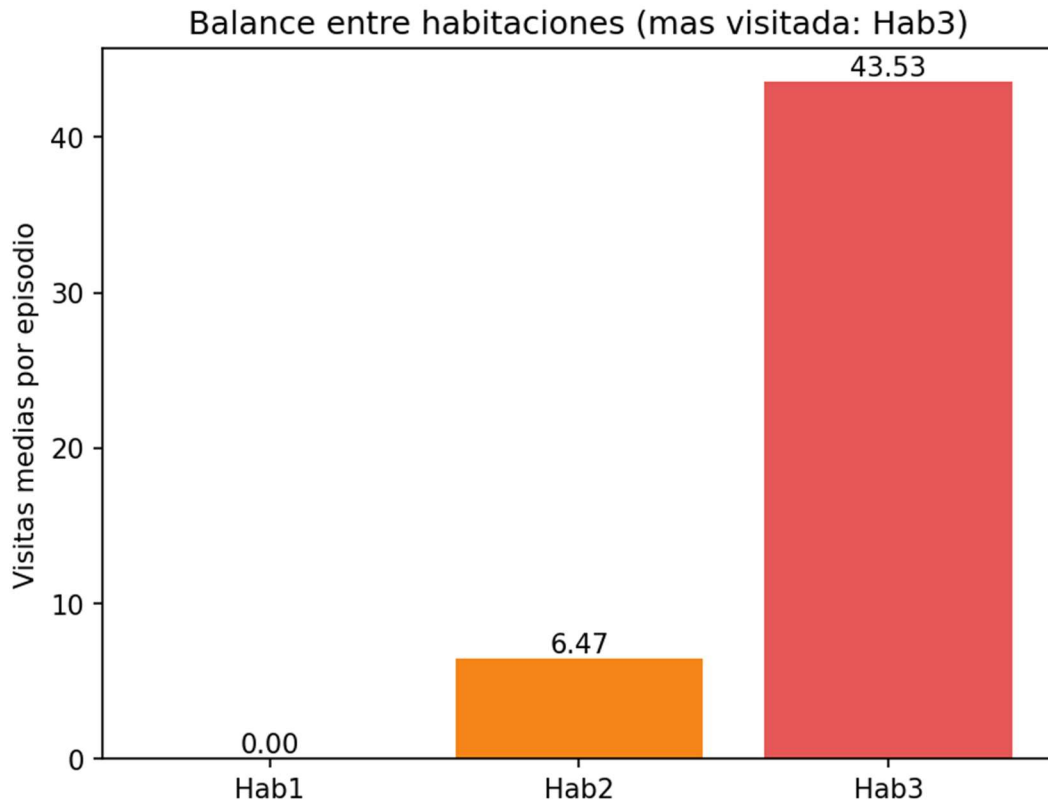


Fig. 19. Balance entre habitaciones exp_009

Los resultados muestran un comportamiento altamente inestable. El agente colapsa hacia una política fuertemente sesgada, ignorando completamente la estación de carga y concentrando la mayoría de las visitas en un subconjunto reducido de habitaciones. La cobertura completa es del 0%, con un desbalance extremo entre habitaciones. La recompensa media por paso alcanza valores elevados, pero este incremento no se traduce en un comportamiento deseable, sino en una amplificación artificial de la señal de recompensa derivada de su naturaleza acumulativa.

En conjunto, este experimento evidencia que la formulación densa basada en desbalance continuo, en su forma actual, no produce una señal de aprendizaje adecuada y motiva el rediseño de la línea de recompensas dispersas en los experimentos posteriores.

5.6. Recompensa dispersa final

La línea de recompensa dispersa surge del diagnóstico de exp_009: si la recompensa se acumula paso a paso, el agente optimiza la señal local en lugar del objetivo global de balance al final del episodio. La solución es desplazar toda la señal de recompensa al final del episodio, eliminando el gradiente intermedio y forzando al agente a razonar sobre el episodio completo.

5.6.1. exp_010: recompensa dispersa de balance al final del episodio

Este experimento establece la formulación base de la línea de recompensa dispersa. Durante los pasos intermedios del episodio, la recompensa se mantiene constante en 0.0. La única señal de refuerzo inmediato corresponde al agotamiento de batería, que genera una penalización de -10.0 con terminación del episodio. En caso de finalización por truncación, se calcula una única recompensa final basada en el grado de balance entre habitaciones, empleando la misma métrica de desviación respecto a la distribución uniforme introducida en exp_009, con un rango de [0, 10].

La hipótesis es que eliminar la señal de recompensa intermedia fuerza al agente a optimizar una métrica global del episodio, evitando la optimización local paso a paso observada en formulaciones anteriores. De este modo, el aprendizaje se centra exclusivamente en maximizar el balance final de visitas.

Esta formulación no incluye penalización explícita por el uso de la estación de carga C, bajo la hipótesis de que su efecto quedaría implícitamente regulado a través de su impacto en la distribución de visitas a habitaciones.

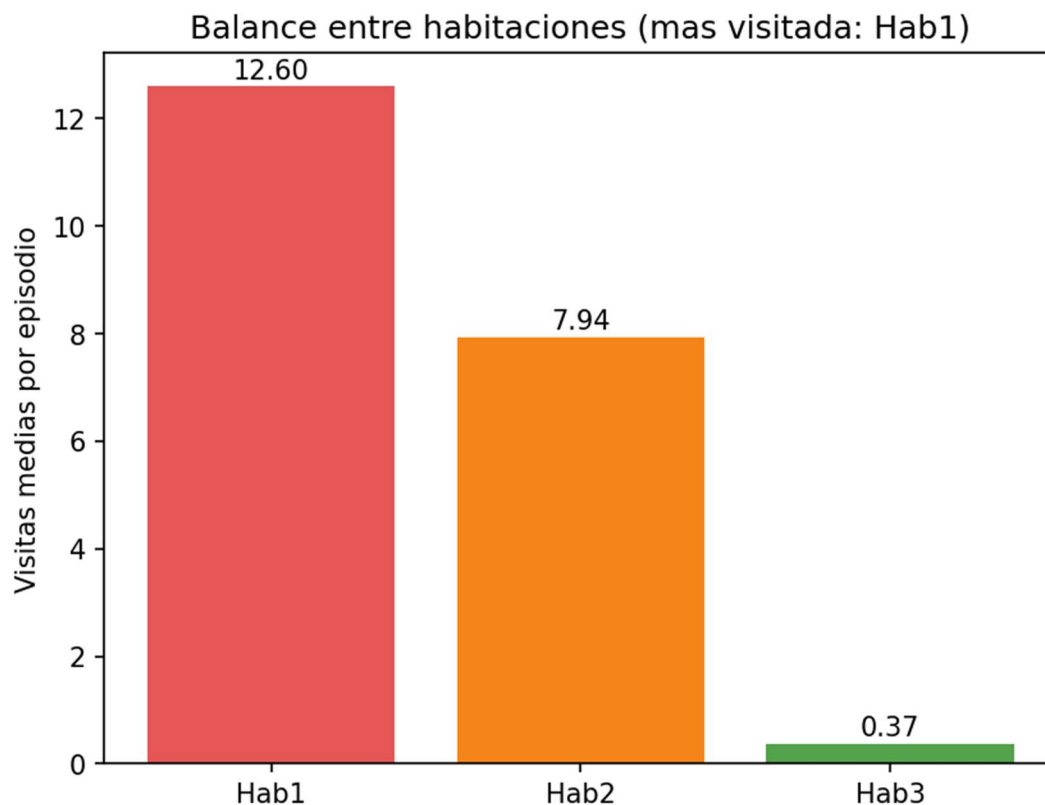


Fig. 20. Balance entre habitaciones exp_010

Los resultados muestran que el agente es capaz de aprender una política de exploración básica, pero con un equilibrio claramente insuficiente. La cobertura completa alcanza el 28.6%, lo que representa una mejora respecto a exp_009, aunque sigue lejos del comportamiento deseado.

Las visitas se concentran de forma desigual entre habitaciones, con 12.60 en Hab1, 7.94 en Hab2 y 0.37 en Hab3, generando un desbalance significativo (12.22 de rango y 5.07 de desviación típica). El tiempo en la estación de carga alcanza el 58.2%, lo que indica

que el agente identifica que la presencia en C no penaliza directamente la recompensa final, ya que no forma parte del cálculo de balance.

En conjunto, estos resultados evidencian que la recompensa dispersa por sí sola no es suficiente para inducir una política de patrullaje completa cuando no existen mecanismos explícitos que regulen el uso de la estación de carga o la distribución mínima de visitas entre habitaciones.

5.6.2. exp_011: recompensa dispersa con penalización cuadrática de C (coeficiente 10)

El undécimo experimento extiende exp_010 incorporando una penalización dispersa final asociada al uso de la estación de carga C. La motivación surge de los resultados anteriores, donde el agente podía utilizar C de forma frecuente sin que ello tuviera un impacto directo sobre la recompensa final.

La penalización se calcula de forma cuadrática sobre la proporción de visitas a C respecto al total de visitas del episodio, según la expresión $penalty_C = -10 \cdot p_C^2$. La recompensa final se obtiene combinando esta penalización con la recompensa de balance entre habitaciones.

La elección de una función cuadrática pretende penalizar de forma creciente los usos excesivos de la estación de carga, manteniendo un impacto reducido cuando las visitas a C son poco frecuentes.

Los resultados muestran que esta modificación no logra corregir el comportamiento observado en exp_010. La cobertura completa permanece en el 0% y el agente continúa desarrollando una política desequilibrada, concentrando las visitas en Hab1 y Hab3 mientras ignora completamente Hab2. Las visitas medias son 13.88 para Hab1, 0.00 para Hab2 y 10.53 para Hab3.

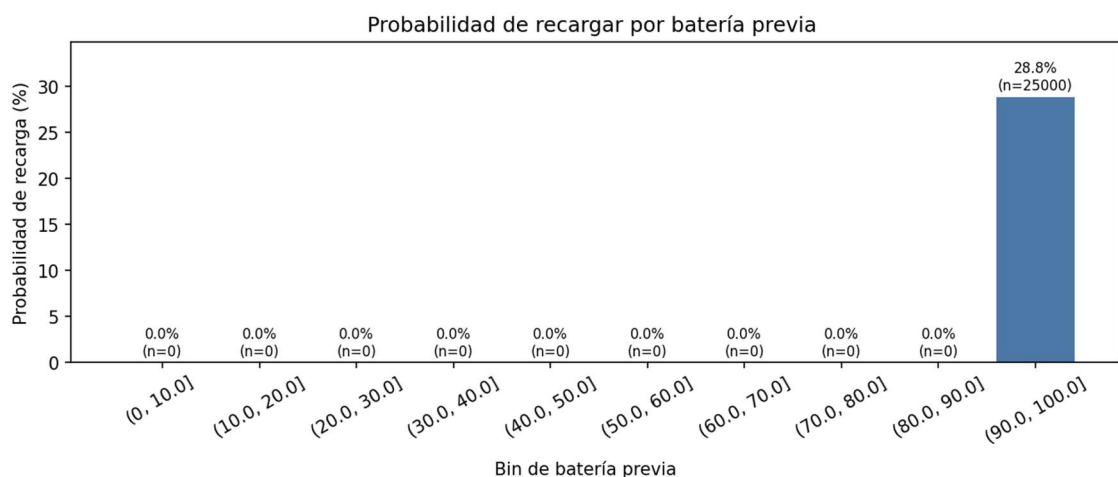


Fig. 21. Probabilidad de recargar exp_011

Además, el tiempo en C aumenta hasta el 51.2% y las cargas medias alcanzan 14.42 por episodio. Este resultado indica que la penalización introducida no tiene suficiente peso relativo para modificar la política aprendida. Aunque la función cuadrática incrementa

el castigo para frecuencias elevadas de uso de C, las penalizaciones obtenidas en la práctica siguen siendo pequeñas en comparación con la recompensa de balance, cuyo valor puede alcanzar hasta 10 puntos.

En conjunto, el experimento muestra que la penalización dispersa aplicada sobre el uso de C no resulta suficiente para corregir los problemas observados en exp_010. La cobertura completa continúa siendo nula, el comportamiento permanece fuertemente sesgado hacia un subconjunto de habitaciones y el uso de la estación de carga sigue siendo elevado. Estos resultados sugieren que es necesario aumentar significativamente el peso de la penalización para que tenga un efecto apreciable sobre la política aprendida.

5.6.3. exp_012: recompensa dispersa con penalización cuadrática de C (coeficiente 50)

El análisis de exp_011 sugirió que la penalización cuadrática con coeficiente 10 resultaba demasiado débil para modificar de forma significativa el comportamiento del agente respecto al uso de la estación de carga. Con valores habituales de p_C , la penalización obtenida era reducida en comparación con la recompensa de balance, por lo que su influencia sobre la política aprendida era limitada.

Este experimento mantiene la misma formulación que exp_011, introduciendo un único cambio: el coeficiente de la penalización cuadrática se incrementa de 10 a 50. Con este escalado, una frecuencia de uso de C de $p_C = 0.2$ genera una penalización de -2.0, mientras que una frecuencia de $p_C = 0.5$ produce una penalización de -12.5. De este modo, el coste asociado a un uso frecuente de la estación de carga pasa a ser comparable e incluso superior a la recompensa máxima de balance entre habitaciones.

Los resultados muestran un cambio significativo respecto a exp_011 en las métricas relacionadas con el uso de C. El tiempo en la estación de carga desciende hasta el 27.5% y el número medio de cargas por episodio se reduce a 10.36, indicando que el agente responde de forma efectiva al aumento de la penalización.

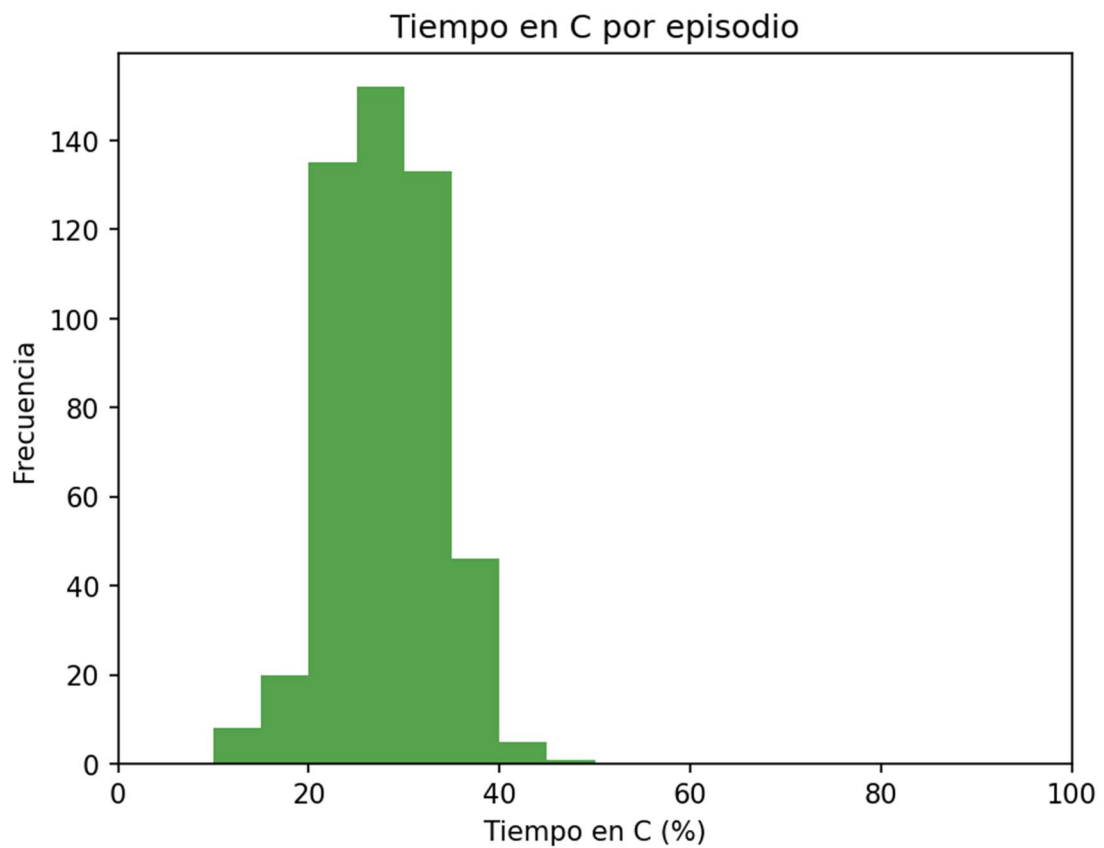


Fig. 22. Tiempo en C por episodio

Sin embargo, esta mejora en la gestión de la estación de carga no se traduce en una política de patrullaje equilibrada. La cobertura completa continúa siendo del 0% y el agente sigue concentrando sus visitas en únicamente dos habitaciones. Las visitas medias son 22.51 para Hab1, 0.00 para Hab2 y 13.75 para Hab3, con un desbalance medio (max-min) de 22.52 y una desviación típica de 9.36.

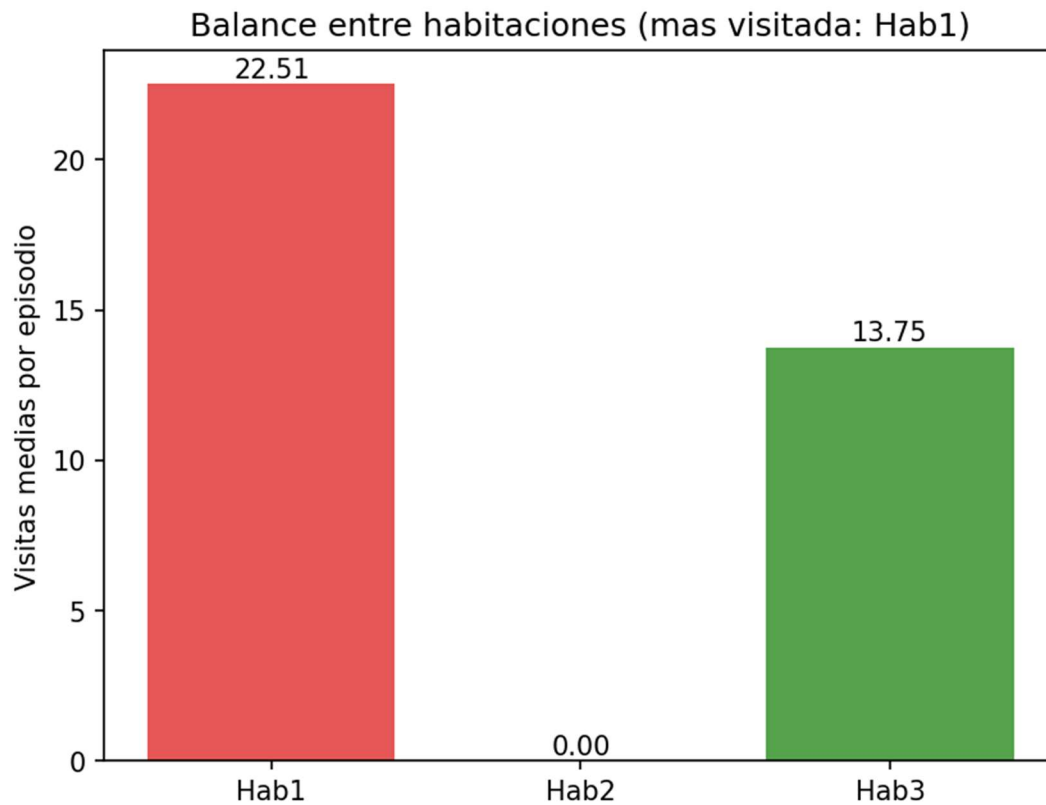


Fig. 23. Balance entre habitaciones exp_012

El aumento del coeficiente de penalización consigue reducir de forma notable el uso de la estación de carga respecto a exp_011, confirmando que el agente responde a cambios en el coste asociado a C. Sin embargo, esta mejora no se traduce en una política de patrullaje equilibrada. El agente continúa ignorando sistemáticamente una de las habitaciones y la cobertura completa permanece en el 0%.

En conjunto, los resultados sugieren que la combinación de recompensa dispersa de balance y penalización del uso de C no resulta suficiente para inducir una exploración completa del entorno. Aunque el comportamiento energético mejora de forma apreciable, persisten problemas estructurales de cobertura que motivan la exploración de modificaciones adicionales en el diseño del entorno.

5.7. Extensión del espacio de acciones: acción stop

Los experimentos de esta línea exploran una modificación estructural del espacio de acciones. Hasta exp_012 el agente solo podía elegir a qué nodo desplazarse. Esta línea introduce una quinta acción, stop, que permite al agente detenerse durante un número de segundos seleccionable. El objetivo es dar al agente la capacidad de gestionar el tiempo dentro del episodio, por ejemplo, pausando la patrulla cuando la batería está en zona segura para evitar visitas innecesarias a C.

El espacio de acciones pasa de Discrete(4) a MultiDiscrete([5, 51]): el primer componente selecciona la acción (0=Hab1, 1=Hab2, 2=Hab3, 3=C, 4=stop) y el segundo selecciona la duración de la parada en segundos (0 a 50), ignorado si la acción no

es stop. La función de recompensa se mantiene igual que en exp_012.

5.7.1. exp_013: acción stop con velocidad acelerada

Este experimento introduce la validación de la nueva acción stop dentro del espacio de acciones ampliado. El objetivo principal no es optimizar el rendimiento del agente, sino comprobar el correcto funcionamiento de la implementación a nivel del entorno, incluyendo la ejecución de la acción de parada, la transición de estados asociados y la correcta gestión del tiempo adicional dentro del episodio.

Durante esta fase el agente opera con una velocidad de ejecución acelerada, con el fin de facilitar la observación del comportamiento dinámico del sistema bajo condiciones de carga más exigentes.

Los resultados de la evaluación muestran que la acción stop se integra correctamente en el entorno y es ejecutada sin errores a nivel de simulación. No obstante, el agente no desarrolla una estrategia clara de uso de esta acción. La cobertura completa alcanza el 72.4%, con visitas medias de 1.23 para Hab1, 14.48 para Hab2 y 18.14 para Hab3, y un desbalance medio (max-min) de 17.19.

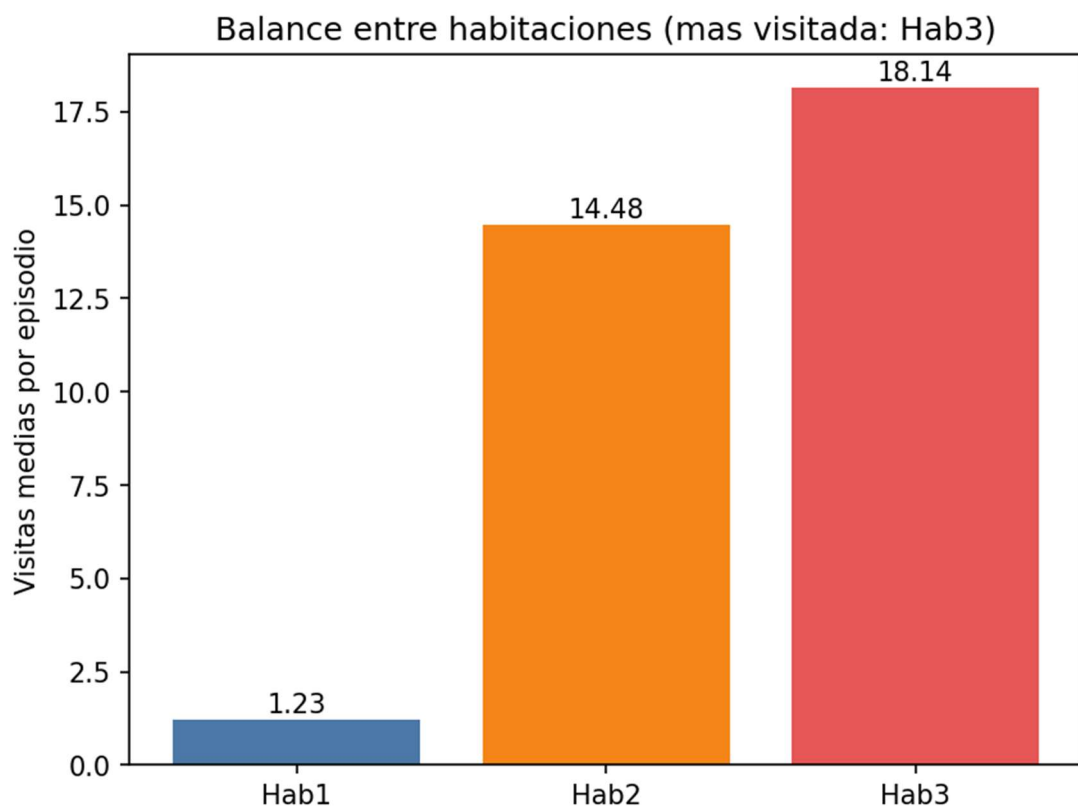


Fig. 24 Balance entre habitaciones exp_013

El tiempo en la estación de carga se sitúa en el 32.3% y las cargas medias en 8.00 por episodio. La duración media de la última acción es de 2.60 segundos y la duración total media del episodio es de 130.11 segundos. La batería mínima media es del 99.5%, lo que indica un comportamiento de recarga muy conservador.

En conjunto, los resultados indican que, aunque la acción stop es funcional desde el punto de vista del entorno, el agente no la incorpora como herramienta efectiva de planificación temporal. La mejora en cobertura respecto a la línea anterior no se acompaña de una estrategia equilibrada de exploración del entorno.

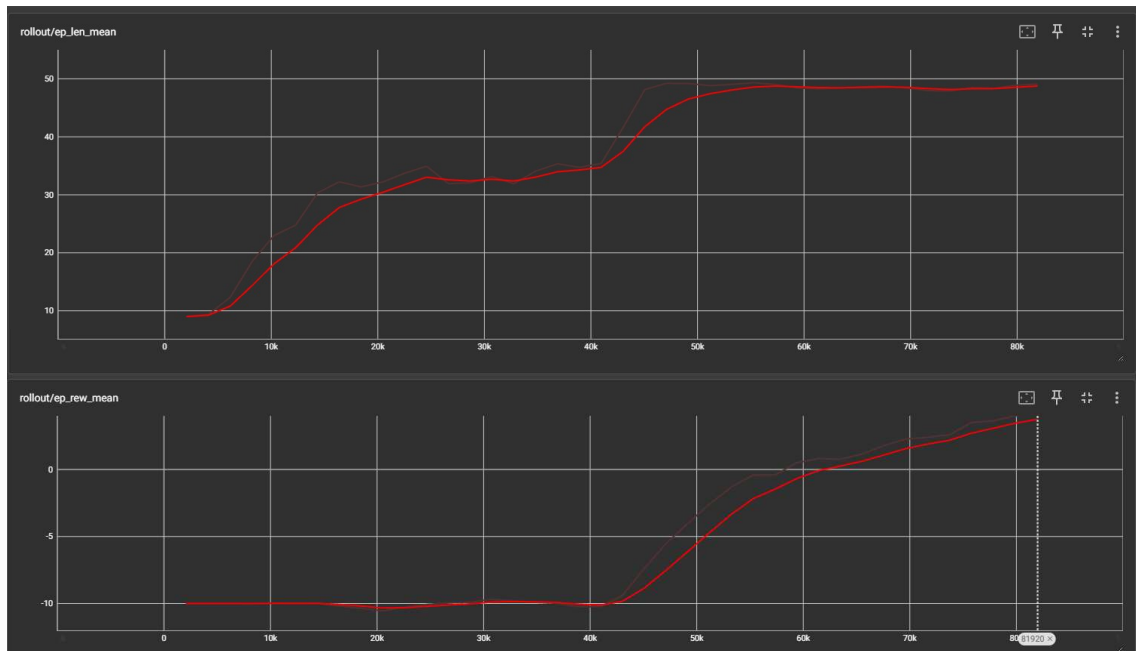
5.7.2. exp_014: acción stop con velocidad normal

Este experimento mantiene la misma formulación de acciones que exp_013, pero restaura la velocidad nominal del robot Pioneer 3-DX (0.5 m/s de velocidad lineal y 1.57 rad/s de velocidad angular). El objetivo principal no es únicamente evaluar el rendimiento final del agente, sino analizar el efecto del cambio de escala temporal sobre la dinámica de aprendizaje y la convergencia de la función de recompensa.

A diferencia de exp_013, en este caso el entorno introduce una mayor duración efectiva de los desplazamientos, lo que incrementa el peso relativo del tiempo de navegación frente a la acción stop. Este cambio altera de forma significativa la relación entre exploración activa y pausa, haciendo que la decisión de detenerse tenga un coste temporal más realista dentro del episodio.

Adicionalmente, durante este experimento se ajustaron los parámetros del modelo de batería con el objetivo de modificar la dinámica de descarga y forzar una mayor presión energética. En particular, se exploraron distintas configuraciones de meseta y fases de caída (incluyendo configuraciones con 20 s y 30 s de meseta en fases iniciales, y configuraciones posteriores con 80 s de meseta, 220 s de caída y 80 s de carga), lo que permitió ajustar el punto en el que la batería entra en fase de descenso efectivo. Estos cambios se realizaron para evitar regímenes en los que la batería permanecía demasiado estable y la acción stop no encontraba relevancia práctica.

El análisis más relevante de este experimento no se encuentra en las métricas finales, sino en la evolución de la curva de entrenamiento de la recompensa media. En la primera mitad del entrenamiento la curva permanece prácticamente plana, sin crecimiento significativo, lo que indica que el agente no está aprovechando la acción stop ni modificando de forma sustancial su política exploratoria. Sin embargo, a partir de la modificación del régimen de batería, la curva deja de ser estacionaria y comienza a mostrar un crecimiento sostenido, indicando un cambio cualitativo en la dinámica de aprendizaje.



Este cambio de régimen sugiere la existencia de un punto de transición en el que la interacción entre duración de episodio, dinámica de batería y coste temporal de navegación hace que la política deje de ser estable y comience a reorganizarse. En este contexto, la acción stop empieza a tener relevancia potencial como mecanismo de control temporal, aunque su uso explícito por parte del agente no se analiza directamente en este experimento.

En términos de evaluación final, el agente alcanza cobertura completa del entorno sin agotamientos de batería, lo que indica estabilidad energética del comportamiento aprendido. No obstante, la distribución de visitas entre habitaciones continúa siendo desigual, lo que sugiere que el cambio en la dinámica temporal afecta principalmente a la eficiencia global del recorrido, pero no corrige completamente el sesgo estructural de exploración.

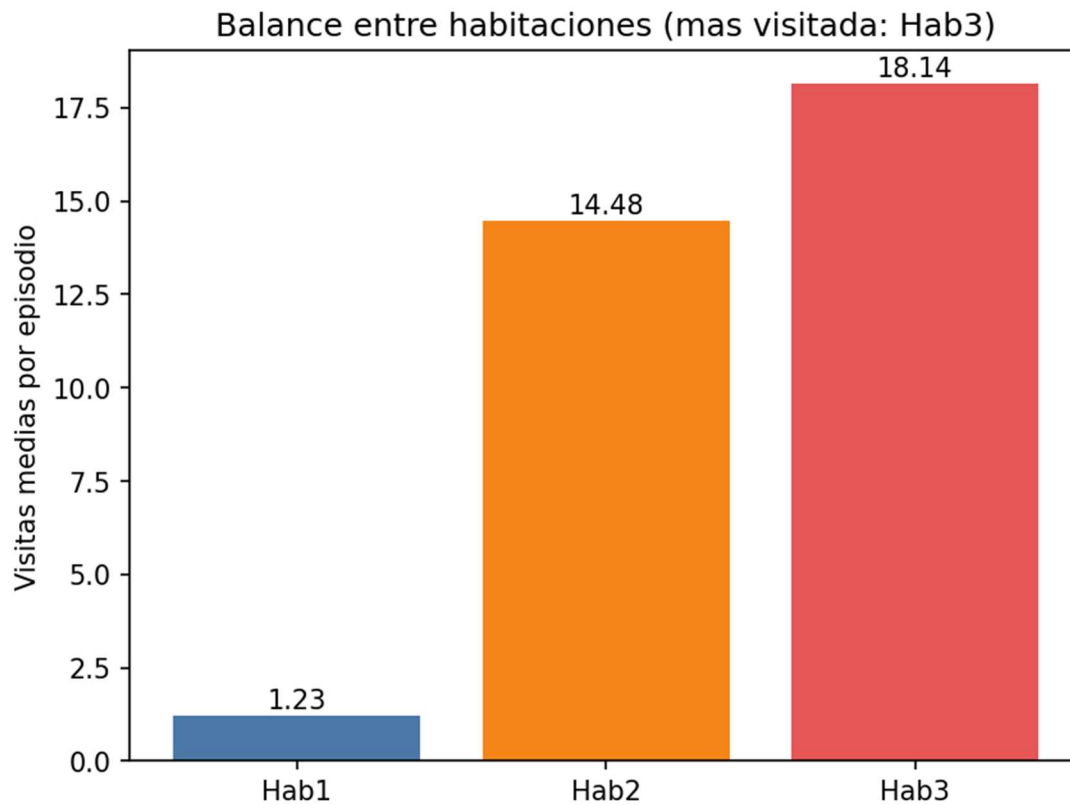


Fig. 25. Balance entre habitaciones exp_014

En conjunto, exp_014 muestra que la restauración de la velocidad nominal, combinada con ajustes en la dinámica de batería, introduce un cambio cualitativo en la convergencia del aprendizaje, evidenciado por la aparición de una fase de crecimiento en la curva de recompensa. Este resultado sugiere que la efectividad de la acción stop depende fuertemente del régimen temporal del entorno, más que de su mera disponibilidad en el espacio de acciones.

5.8. Información temporal en el agente

Las líneas anteriores exploraron la función de recompensa y el espacio de acciones manteniendo constante la observación del agente. Esta línea aborda una dimensión diferente: si el agente dispone de información temporal explícita, ¿puede aprender políticas más eficientes sin necesidad de modificar la recompensa?

Hasta exp_014 el agente solo conoce su posición, el nivel de batería y los contadores de visitas. No tiene acceso a ninguna señal que le indique en qué punto del episodio se encuentra ni cuánto tiempo ha consumido cada acción. Esta línea estudia si añadir esa información a la observación cambia el comportamiento aprendido.

Los cuatro experimentos de esta línea comparten la misma función de recompensa dispersa con penalización cuadrática de C de exp_012, variando únicamente la composición del vector de observación y, en el caso de exp_017, la función de recompensa.

5.8.1. exp_015: validación de SIM_ACCELERATION=3.0 (test de regresión)

Este experimento tiene como objetivo validar que la introducción del factor de aceleración de simulación SIM_ACCELERATION=3.0 no altera la dinámica de aprendizaje del agente respecto a configuraciones previas sin aceleración. El propósito no es evaluar el rendimiento del agente en términos de comportamiento, sino comprobar la equivalencia del proceso de convergencia bajo un escalado temporal distinto del entorno.

El mecanismo de aceleración modifica exclusivamente la relación entre tiempo de simulación y tiempo real, de forma que el entorno evoluciona más rápidamente sin alterar la secuencia de estados observados por el agente ni la dinámica interna del modelo de batería. Bajo esta condición, la hipótesis es que la curva de recompensa en entrenamiento debe conservar su forma global independientemente de la aceleración aplicada.

El análisis se centra en la evolución de la recompensa media durante el entrenamiento. Los resultados muestran que la curva es consistente con la observada en exp_013, manteniendo el mismo patrón de exploración inicial seguido de una fase de estabilización progresiva. No se observan desplazamientos en el punto de convergencia ni cambios relevantes en la tendencia general del aprendizaje, lo que indica que la política evoluciona de forma equivalente bajo ambas configuraciones.

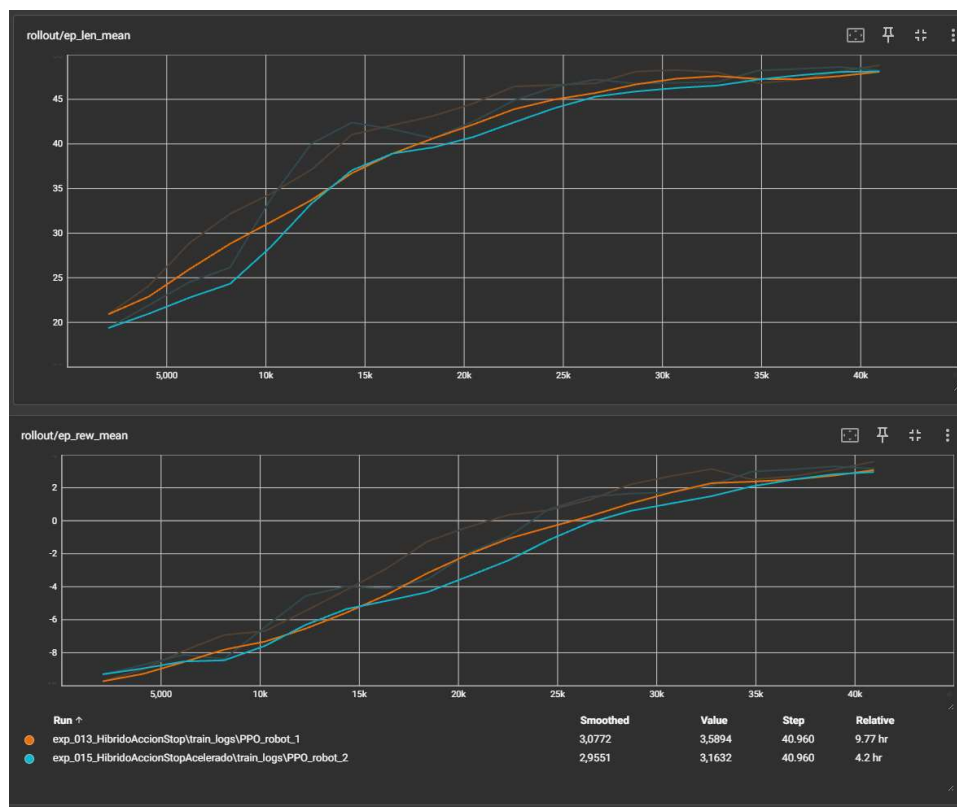


Fig. 26. Comparación curva exp_013 y exp_015

La principal diferencia observada se encuentra en la eficiencia computacional, ya que la aceleración reduce significativamente el tiempo de entrenamiento en tiempo real sin afectar al número de interacciones necesarias para la convergencia. Esto permite mantener la misma dinámica de aprendizaje con un menor coste de simulación.

En consecuencia, SIM_ACCELERATION=3.0 queda validado como configuración estable para el resto de los experimentos, al no introducir desviaciones apreciables en la curva de aprendizaje ni en el proceso de convergencia del modelo.

5.8.2. exp_016: tiempo de episodio en la observación

Este experimento incorpora una nueva componente al vector de observación: tiempo_ep, que representa el progreso normalizado del episodio en el rango [0, 1], donde 0 corresponde al inicio y 1 al final del episodio. En consecuencia, el espacio de observación pasa de seis a siete componentes.

La hipótesis es que esta señal permite al agente diferenciar explícitamente entre fases del episodio, introduciendo contexto temporal en la toma de decisiones. Esto puede modificar la política aprendida sin alterar la función de recompensa, permitiendo comportamientos diferenciados entre etapas tempranas (exploración) y finales (finalización de cobertura y gestión energética).

Este experimento se plantea como una ablación estricta respecto a exp_015, siendo la única modificación la inclusión de la variable tiempo_ep. Por tanto, cualquier cambio en el comportamiento observado puede atribuirse directamente a esta señal temporal.

Los resultados muestran que la incorporación del progreso temporal produce cambios relevantes principalmente en la gestión energética del agente. La cobertura completa se mantiene en el 100% sin agotamientos de batería, lo que indica estabilidad del aprendizaje. Sin embargo, se observa una reducción significativa del nivel de batería en fases de operación, con una batería mínima media del 71.4% y un percentil 10 del 53%, lo que refleja un uso más activo de la energía en comparación con configuraciones anteriores.

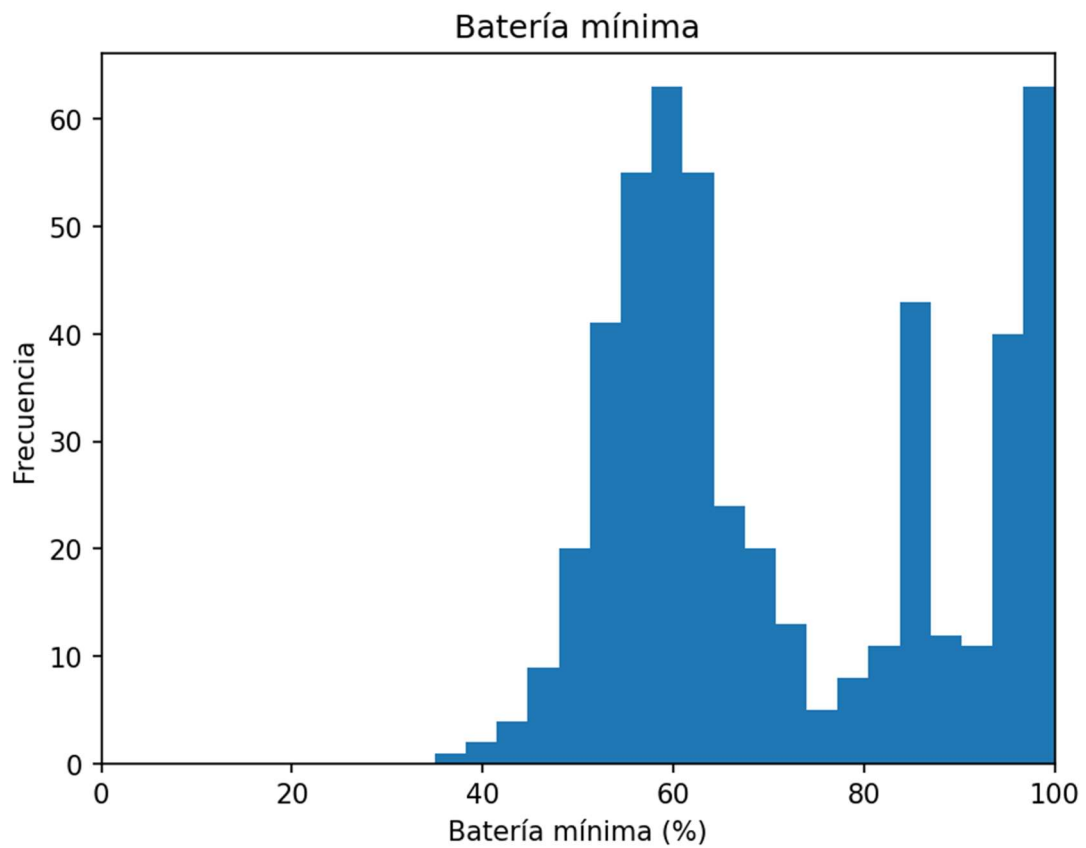


Fig. 27. Batería mínima exp_016

La batería media al recargar se sitúa en el 94.3%, lo que indica que el agente reduce la tendencia a recargas preventivas y adopta un comportamiento más dependiente del estado real de energía. En contraste, el balance entre habitaciones no muestra una mejora sustancial, manteniendo una desviación típica de 3.04 y un rango de 7.04, con un sesgo persistente hacia Hab1.

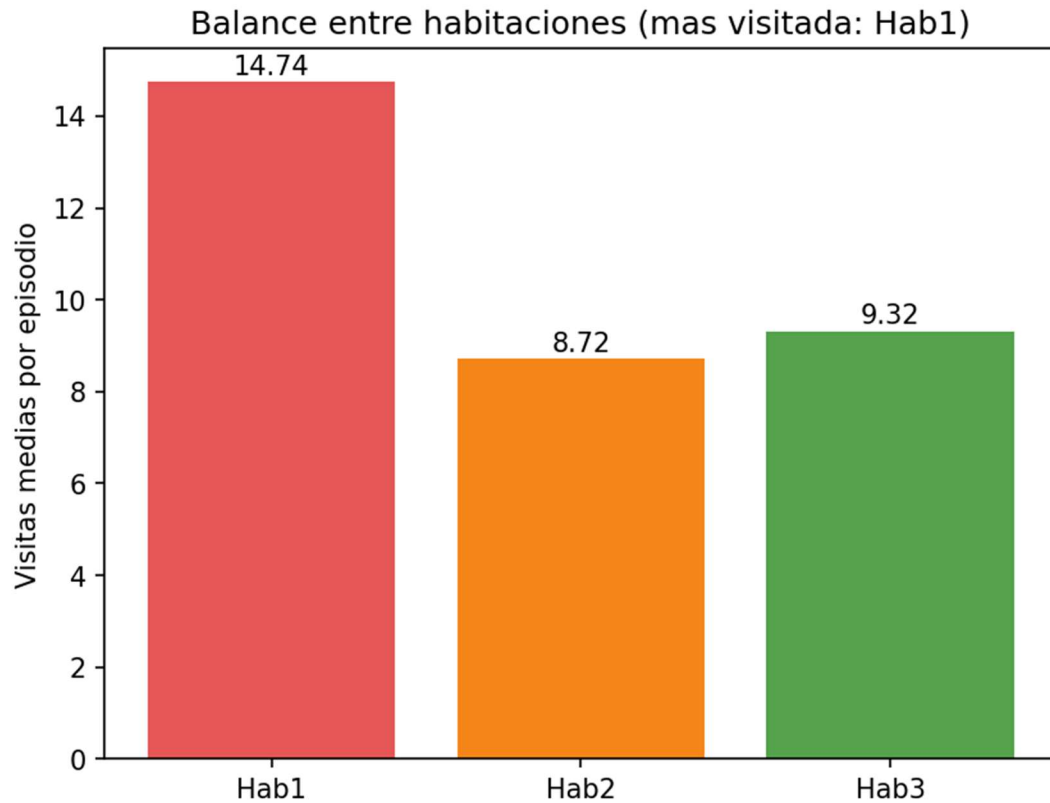


Fig. 28. Balance entre habitaciones exp_016

En conjunto, los resultados indican que la señal de progreso temporal influye principalmente en la gestión energética del agente, mientras que no es suficiente por sí sola para corregir el sesgo estructural en la exploración del entorno.

5.8.3. exp_017: tiempo de episodio en observación y presión temporal en recompensa

Este experimento extiende exp_016 incorporando una presión temporal suave en la función de recompensa. Además de incluir tiempo_ep en la observación, se introduce una penalización pequeña por paso no terminal con el objetivo de incentivar una finalización más eficiente de la cobertura y reducir posibles ineficiencias temporales en forma de esperas o acciones redundantes.

La hipótesis plantea que la combinación de tiempo como contexto (observación) y tiempo como coste (recompensa) podría mejorar la eficiencia global del episodio sin comprometer la estabilidad energética ni el balance entre habitaciones. En particular, se espera una reducción del número de pasos necesarios para alcanzar la cobertura completa, manteniendo un comportamiento de exploración estable.

Este experimento permite contrastar dos roles distintos de la información temporal: como señal contextual (exp_016) frente a como incentivo directo (exp_017), evaluando si la penalización temporal aporta mejoras o introduce sesgos en la política aprendida.

Los resultados muestran que la introducción de presión temporal no mejora la eficiencia de cobertura respecto a exp_016. La cobertura completa se mantiene en el 100% sin agotamientos de batería, pero el paso medio de cobertura completa aumenta ligeramente hasta 27.3, lo que indica que la penalización por paso no solo no acelera la cobertura, sino que tampoco modifica de forma efectiva la dinámica de finalización del episodio.

En términos energéticos, se observa un cambio relevante en el régimen de recarga. Aunque la batería se mantiene en valores seguros, el agente adopta un comportamiento altamente conservador, recargando sistemáticamente con batería máxima (100% de media al recargar). Este efecto sugiere que la penalización temporal desplaza la política hacia una estrategia de seguridad energética excesiva, en lugar de mejorar la eficiencia temporal.

El tiempo en C desciende al 27.8% y las cargas medias a 9.20, lo que indica una ligera mejora en el uso de la estación de carga, aunque no se traduce en una mejora en la velocidad de cobertura ni en el balance entre habitaciones. El desbalance permanece elevado, con desviación típica de 3.38 y rango de 7.91, manteniendo un sesgo claro hacia Hab1.

Este cambio en la política energética se aprecia en la distribución del primer punto de recarga:

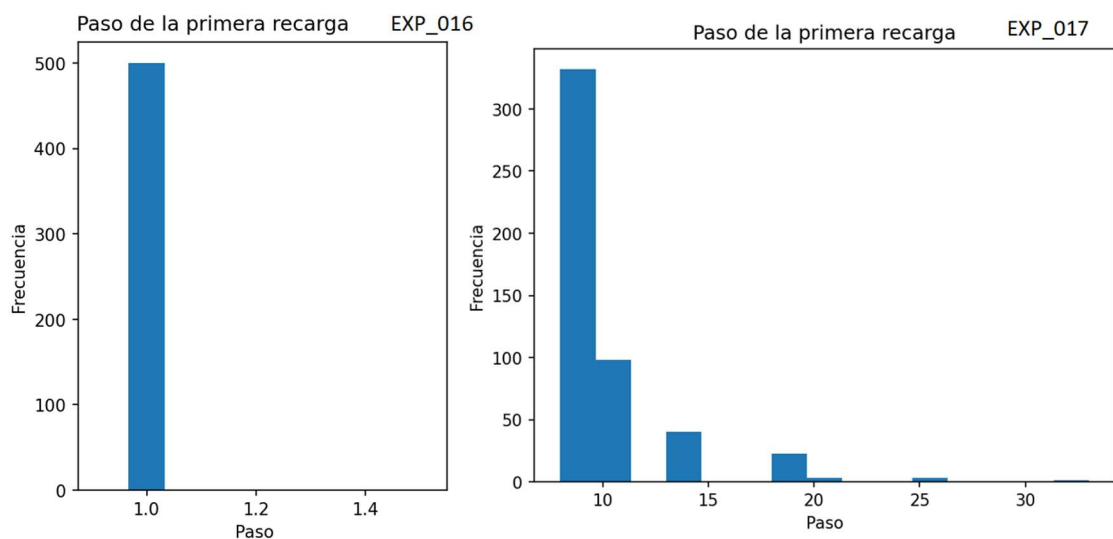


Fig. 29. Comparación primera recarga Exp_016 y Exp_017

En conjunto, exp_017 muestra que la incorporación de tiempo como señal de recompensa no mejora la eficiencia global del sistema y tiende a inducir un comportamiento más conservador en la gestión energética, sin resolver el problema estructural de balance entre habitaciones.

5.8.4. exp_018: duración real de la última acción en la observación

El último experimento de esta línea sustituye la señal de progreso del episodio `tiempo_ep` por una señal alternativa: `tiempo_ultima_accion_s`, que representa la duración real en segundos de la última acción ejecutada. Esta duración se mide en tiempo virtual de simulación y se incorpora como la última componente del vector de observación.

La motivación de este experimento difiere de `exp_016` y `exp_017`. En lugar de proporcionar información global sobre el progreso del episodio, se introduce una señal local centrada en el coste temporal inmediato de las acciones. Esta distinción es especialmente relevante en el caso de la acción `stop`, cuya duración es variable y seleccionada por el propio agente, así como en los desplazamientos entre nodos, donde el tiempo de ejecución depende de la distancia recorrida.

La señal se obtiene en el entorno `CoppeliaSim` mediante `virtualTime` y se publica como `robot_last_action_duration`, siendo posteriormente incorporada al vector de observación desde Python a través del wrapper del entorno. Al utilizar tiempo virtual, la medida se mantiene independiente de `SIM_ACCELERATION`, garantizando coherencia entre configuraciones experimentales.

La hipótesis es que esta información permitirá al agente diferenciar entre acciones de alto y bajo coste temporal, favoreciendo la selección de secuencias más eficientes sin necesidad de introducir penalizaciones explícitas en la función de recompensa ni depender de contadores globales de pasos.

Los resultados muestran un comportamiento globalmente estable, pero con efectos mixtos. La cobertura completa se mantiene en el 100% sin agotamientos de batería. El tiempo en C desciende al 28.8% y las cargas medias son de 9.81, mientras que la recompensa por paso media se sitúa en 0.088. En términos energéticos, el agente mantiene un comportamiento conservador, con batería mínima media del 98.0% y percentil 10 del 95%, similar a `exp_017`.

El paso medio de cobertura completa alcanza 22.3, el mejor valor de toda la línea temporal, lo que sugiere una mejora en la eficiencia de finalización del episodio. Sin embargo, esta mejora no se traslada al balance entre habitaciones, que empeora respecto a `exp_016` y `exp_017`, con una desviación típica de 5.07 y un rango de 12.03. Las visitas se distribuyen de forma desigual, con 12.09 en Hab1, 5.83 en Hab2 y 17.67 en Hab3.

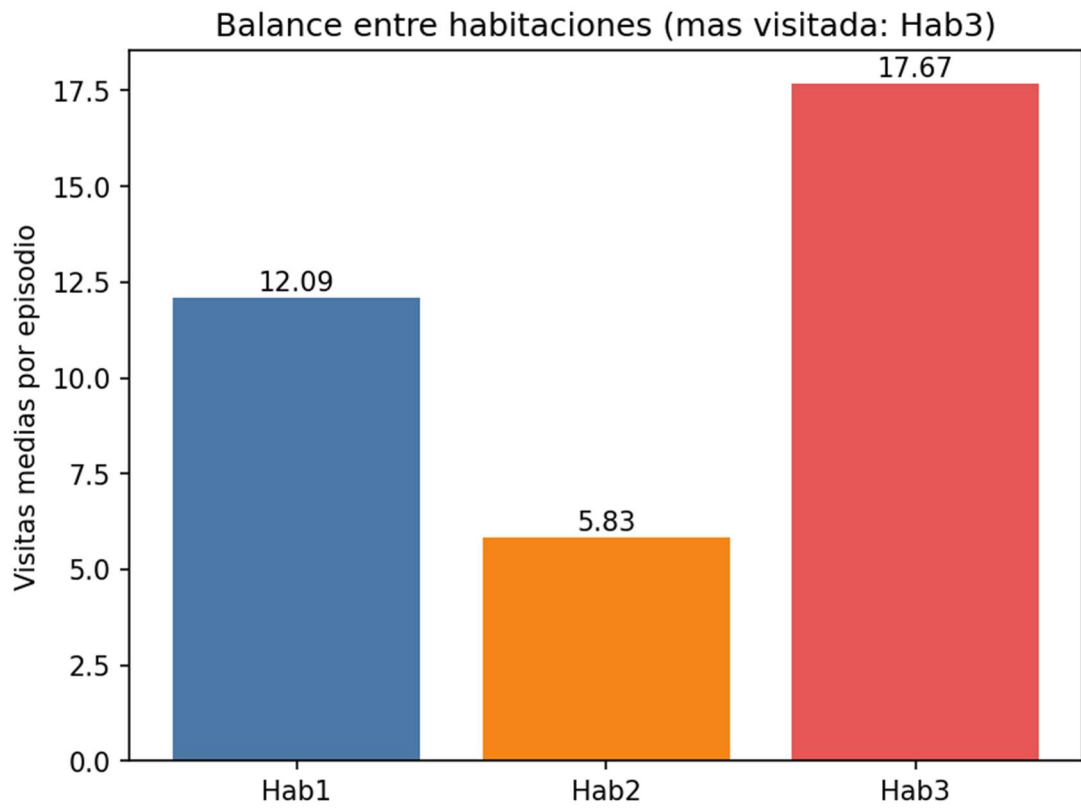


Fig. 30. Balance entre habitaciones exp_018

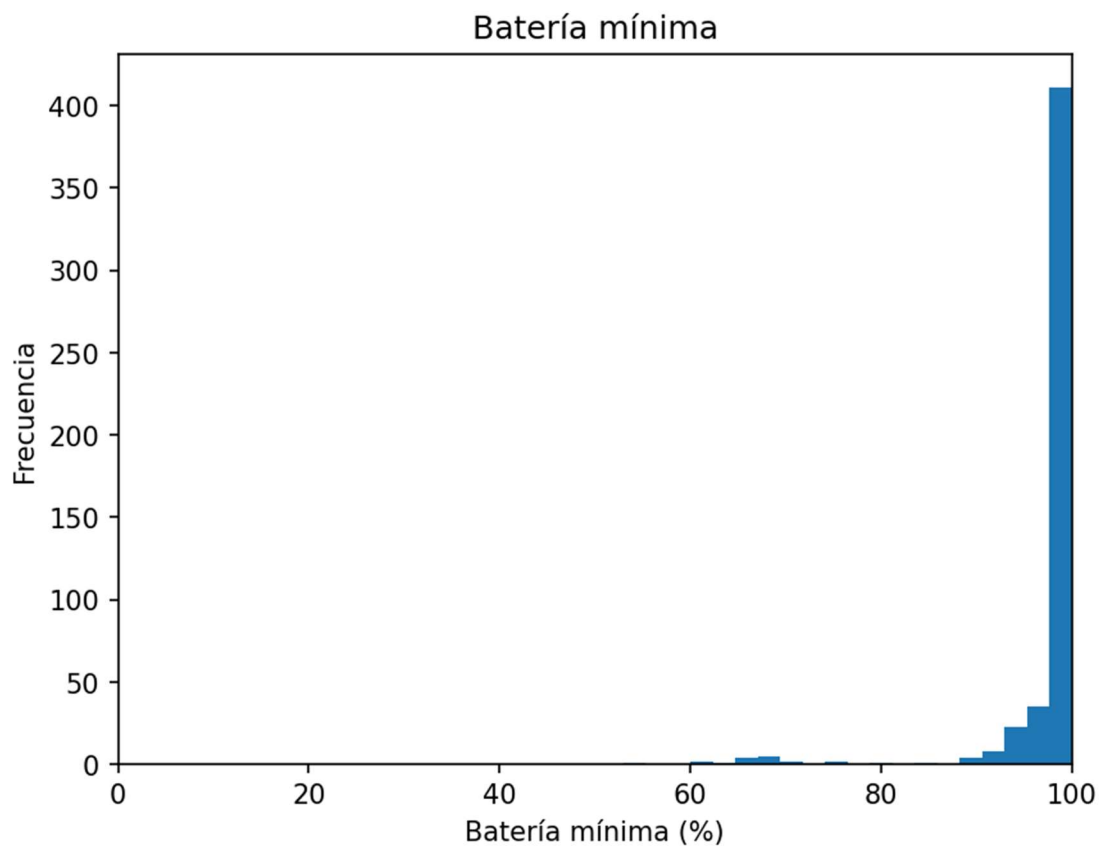


Fig. 31. Batería mínima exp_018

En conjunto, exp_018 indica que la información sobre la duración de la última acción aporta mejoras puntuales en la eficiencia temporal del episodio, pero no resulta suficiente para corregir el sesgo estructural en la exploración del entorno, ni mejora de forma consistente la política de balance entre habitaciones.

5.9. Comparativa final

La siguiente tabla resume las métricas clave de los dieciocho experimentos evaluados sobre 500 episodios con política determinista:

Experimento	Cob.	Bat.0	T.C	Cargas	R/paso	Std	Rango
exp_001_remake	0%	0	100%	1.00	0.000	0.00	0.00
exp_002	0%	0	67.9%	13.31	0.320	4.89	11.71
exp_003	100%	0	48.4%	16.25	0.514	1.97	4.58
exp_004	100%	0	51.2%	18.03	0.270	1.29	2.99
exp_005	100%	0	29.7%	10.10	0.365	1.61	3.75
exp_006	100%	0	26.0%	11.84	0.378	1.29	2.98
exp_007	100%	0	27.0%	11.00	0.323	1.30	3.00
exp_008	100%	0	38.4%	14.50	0.333	1.84	4.28
exp_009	0%	0	0.0%	0.00	4.488	19.18	43.53
exp_010	28.6%	0	58.2%	3.48	0.148	5.07	12.22
exp_011	0%	0	51.2%	14.42	0.093	5.99	14.00
exp_012	0%	0	27.5%	10.36	0.052	9.36	22.52
exp_013	72.4%	0	32.3%	8.00	0.055	7.42	17.19
exp_014	100%	0	20.4%	8.80	0.131	6.93	16.69
exp_015	100%	0	32.7%	11.59	0.072	5.33	12.67
exp_016	100%	0	34.4%	7.99	0.083	3.04	7.04
exp_017	100%	0	27.8%	9.20	0.091	3.38	7.91
exp_018	100%	0	28.8%	9.81	0.088	5.07	12.03

El análisis global de los dieciocho experimentos permite extraer varias conclusiones estructuradas por líneas de diseño.

La primera línea experimental (exp_001 a exp_008) es la que proporciona los resultados globalmente más equilibrados del problema. Dentro de ella, la penalización adaptativa introducida en exp_005 supone el avance más significativo, y exp_006 representa la configuración más estable en términos conjuntos de cobertura, eficiencia energética y balance entre habitaciones. Este último es el único experimento que satisface simultáneamente los tres bloques de criterios de éxito definidos.

La segunda línea (exp_009 a exp_012) muestra que las formulaciones de recompensa dispersa no son suficientes por sí mismas para inducir una política de patrullaje estable. exp_009 presenta un colapso completo de la cobertura debido a la naturaleza de la señal acumulativa, mientras que exp_010 a exp_012 consiguen mejorar parcialmente el uso de la estación de carga, pero sin lograr una exploración completa ni un balance consistente entre habitaciones. El problema principal de esta línea es la falta de un

gradiente intermedio suficientemente informativo para guiar la política durante el episodio.

La tercera línea (exp_013 y exp_014) demuestra que la introducción de la acción stop es técnicamente funcional y que la restauración de la velocidad nominal en exp_014 reduce de forma notable el tiempo en la estación de carga. Sin embargo, este efecto se produce a costa de un deterioro significativo del balance entre habitaciones, lo que indica que la capacidad de control temporal por sí sola no induce una política espacial equilibrada.

La cuarta línea (exp_015 a exp_018) evidencia que la incorporación de información temporal en la observación mejora aspectos concretos de la gestión energética y de la eficiencia temporal del episodio, pero no resuelve el problema estructural de distribución de visitas. En particular, exp_016 reduce la tendencia a recargas preventivas, pero todas las variantes mantienen desviaciones de balance elevadas, con valores significativamente superiores a los obtenidos en la primera línea experimental.

En conjunto, el mejor resultado global del sistema corresponde a exp_006, con cobertura del 100%, tiempo en C del 26.0%, cargas medias de 11.84, desviación típica de balance de 1.29 y rango de 2.98. Este experimento constituye la única configuración que satisface simultáneamente los tres bloques de criterios de éxito definidos en el capítulo 4.

6

Conclusiones y líneas futuras

6.1. Conclusiones

El objetivo principal de este Trabajo de Fin de Grado era desarrollar un sistema de control de alto nivel basado en aprendizaje por refuerzo capaz de gestionar de forma autónoma la cobertura de un entorno interior y la autonomía energética de un robot móvil simulado. Los resultados obtenidos permiten concluir que dicho objetivo se ha alcanzado satisfactoriamente.

En primer lugar, se diseñó e implementó correctamente un entorno de simulación completo en CoppeliaSim, integrando el robot Pioneer P3DX, un sistema de navegación basado en grafos y un modelo de batería configurable. La comunicación síncrona mediante ZeroMQ permitió controlar el simulador desde Python y definir el problema como un entorno compatible con Gymnasium, facilitando la integración con Stable-Baselines3 y el entrenamiento de agentes PPO.

El trabajo realizado demuestra además que el diseño de la función de recompensa es el factor más determinante en el comportamiento emergente del agente. Las primeras configuraciones experimentales evidenciaron comportamientos degenerados, como permanecer permanentemente en la estación de carga o ignorar determinadas habitaciones. A partir de estas observaciones se desarrolló un proceso iterativo de refinamiento que permitió introducir penalizaciones adaptativas y criterios de balance más robustos.

Los resultados experimentales muestran que la línea de recompensas densas basada en penalización adaptativa fue la más efectiva. En particular, el experimento exp_006 obtuvo el mejor equilibrio global entre cobertura completa, estabilidad energética y balance entre habitaciones, siendo el único experimento que cumplió

simultáneamente todos los bloques de criterios de evaluación definidos en la metodología. Este resultado confirma que una señal de recompensa cuidadosamente diseñada puede inducir comportamientos complejos de patrullaje sin necesidad de programar reglas explícitas.

Por otro lado, las líneas basadas exclusivamente en recompensas dispersas mostraron mayores dificultades de convergencia y balance, incluso incorporando penalizaciones cuadráticas por el uso excesivo de la estación de carga. Esto sugiere que, en problemas de control estratégico de larga duración, la ausencia total de señal intermedia dificulta significativamente el aprendizaje estable.

La ampliación del espacio de acciones mediante la acción stop permitió comprobar que el agente es capaz de incorporar decisiones temporales adicionales, aunque no se observaron mejoras claras en el balance de patrulla. Del mismo modo, la incorporación de información temporal en la observación modificó el comportamiento energético y la estrategia aprendida, especialmente en exp_016, donde el agente desarrolló políticas menos conservadoras respecto al uso de batería. Sin embargo, estas señales temporales no resolvieron completamente el problema del desequilibrio entre habitaciones.

A nivel técnico, también se validó el uso de `SIM_ACCELERATION=3.0` como mecanismo para reducir significativamente el tiempo de entrenamiento sin alterar la dinámica del aprendizaje ni las métricas finales de evaluación. Este resultado tiene especial relevancia práctica, dado el elevado coste computacional asociado al entrenamiento de agentes de aprendizaje por refuerzo en simulación robótica.

En conjunto, el trabajo demuestra que el aprendizaje por refuerzo es una herramienta viable para problemas de control de alto nivel en robótica móvil, especialmente cuando se dispone de una representación clara del entorno y de criterios de recompensa bien definidos. Asimismo, pone de manifiesto la importancia del diseño experimental y de la evaluación sistemática en este tipo de sistemas, donde pequeñas modificaciones en la función de recompensa pueden producir cambios drásticos en el comportamiento aprendido.

6.2. Líneas futuras

Existen múltiples líneas de trabajo que podrían ampliarse a partir del sistema desarrollado en este proyecto.

Una de las más relevantes sería incrementar la complejidad del entorno de simulación. El sistema actual utiliza un grafo reducido con un número limitado de habitaciones y rutas totalmente conocidas. Una extensión natural consistiría en utilizar mapas más grandes, con obstáculos dinámicos, rutas no completamente conexas o múltiples estaciones de carga, aproximando el problema a escenarios reales de vigilancia autónoma.

También resultaría interesante incorporar incertidumbre en la percepción y en la navegación del robot. En el sistema actual, el desplazamiento entre nodos se considera

siempre exitoso y determinista. Introducir ruido en sensores, errores de localización o fallos de navegación permitiría evaluar la robustez de las políticas aprendidas en condiciones menos ideales.

Otra línea de trabajo relevante sería estudiar algoritmos distintos de PPO. Aunque PPO ha mostrado un comportamiento estable y adecuado para este problema, podrían evaluarse algoritmos off-policy más eficientes en muestras, arquitecturas recurrentes con memoria temporal o enfoques jerárquicos capaces de separar explícitamente planificación estratégica y control táctico.

Desde el punto de vista de la representación del estado, una posible mejora consistiría en incorporar memoria histórica o mecanismos de atención que permitan al agente modelar dependencias temporales más largas. Los resultados obtenidos sugieren que la información temporal influye en el comportamiento aprendido, pero las señales utilizadas en este trabajo son todavía relativamente simples.

También podría profundizarse en el estudio de recompensas multiobjetivo y técnicas de reward shaping más avanzadas. El trabajo ha mostrado que pequeños cambios en la recompensa producen comportamientos muy distintos, por lo que sería interesante explorar métodos automáticos de ajuste de recompensas o aprendizaje inverso de recompensas.

Otra posible ampliación sería trasladar el sistema desde simulación a un robot físico real. Aunque el trabajo se ha desarrollado íntegramente en CoppeliaSim, la arquitectura modular basada en Gymnasium y ZeroMQ facilitaría adaptar la capa de control a plataformas robóticas reales compatibles con ROS o sistemas equivalentes.

Finalmente, sería interesante explorar escenarios multiagente donde varios robots cooperen para patrullar el entorno de forma coordinada, compartiendo recursos energéticos y distribuyendo dinámicamente las zonas de vigilancia. Este tipo de problemas introduce nuevos desafíos relacionados con coordinación, comunicación y reparto eficiente de tareas.

Referencias

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford y O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [En línea]. Disponible en: <https://arxiv.org/abs/1707.06347>
- [2] Y. Chevaleyre, "Theoretical analysis of the multi-agent patrolling problem," en *Proc. IEEE/WIC/ACM International Conference on Intelligent Agent Technology*, 2004, pp. 302–308.
- [3] Z. Huang, H. Liu y C. Su, "Multi-Agent Patrol Policy Learning with Deep Reinforcement Learning," en *Proc. IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2019.
- [4] G. Notomista, S. Mayya, A. Mazumdar, S. Hutchinson y M. Egerstedt, "A Resilient and Energy-Aware Task Allocation Framework for Heterogeneous Multi-Robot Systems," *IEEE Transactions on Robotics*, 2020.
- [5] M. Morales, *Grokking Deep Reinforcement Learning*. Manning, 2020.
- [6] Farama Foundation, "Gymnasium Documentation." [En línea]. Disponible en: <https://gymnasium.farama.org/>
- [7] Stable-Baselines3, "Stable-Baselines3 Documentation." [En línea]. Disponible en: <https://stable-baselines3.readthedocs.io/>
- [8] Coppelia Robotics, "CoppeliaSim User Manual." [En línea]. Disponible en: <https://www.coppeliarobotics.com/helpFiles/>
- [9] PyTorch, "PyTorch Documentation." [En línea]. Disponible en: <https://pytorch.org/docs/stable/>
- [10] Google, "TensorBoard Documentation." [En línea]. Disponible en: <https://www.tensorflow.org/tensorboard>
- [11] Matplotlib Development Team, "Matplotlib Documentation." [En línea]. Disponible en: <https://matplotlib.org/stable/>
- [12] Lua.org, "Lua 5.4 Reference Manual." [En línea]. Disponible en: <https://www.lua.org/manual/5.4/>
- [13] Hugging Face, "Deep RL Course." [En línea]. Disponible en: <https://huggingface.co/learn/deep-rl-course/unit0/introduction>

Apéndice A. Manual de Instalación

A.1. Requerimientos

- Sistema operativo: Windows 10/11 (el repositorio incluye scripts .bats orientados a Windows). El proyecto también puede ejecutarse en Linux/Mac modificando las rutas y los scripts de arranque.
- CoppeliaSim Edu (versión compatible con la API ZeroMQ Remote). Instale CoppeliaSim desde la web oficial y anote la ruta de instalación. Los scripts del repositorio asumen por defecto:

C:\Program Files\CoppeliaRobotics\CoppeliaSimEdu\coppeliaSim.exe

Si su instalación está en una ruta distinta, actualice los ficheros en `scripts/` o los .bats correspondientes.

- Python 3.8 - 3.11 (cualquier versión dentro de este rango es compatible con las dependencias usadas).
- Dependencias Python: todas las dependencias necesarias están listadas en auxiliares/requirements.txt. Instálelas con pip:

Instale las dependencias con:

```
``bash
python -m pip install --upgrade pip
pip install -r auxiliares/requirements.txt
``
```

La lista contenida en `auxiliares/requirements.txt` incluye, entre otros, los siguientes paquetes clave: `stable-baselines3` (con backend PyTorch), `gymnasium`, `coppeliasim\_zmqremoteapi\_client`, `numpy`, `pandas`, `matplotlib`, `tqdm`, `pyzmq`, `torch`, `tensorboard` y `pillow`.

Nota: `stable-baselines3` descargará sus dependencias adicionales automáticamente al instalarse (incluyendo dependencias de PyTorch si no están presentes).

- -- CoppeliaSim ZeroMQ Remote API: el proyecto utiliza la integración ZeroMQ. Asegúrese de que CoppeliaSim está configurado para permitir conexiones remotas y de que la librería cliente `coppeliasim\_zmqremoteapi\_client` esté disponible. Si la librería no se instala desde `requirements.txt`, instálela manualmente según la documentación de CoppeliaSim.

A.2. Instalación y configuración

1. Coloque la escena `entornoRobotVigilancia.ttt` en la instalación de CoppeliaSim o abra la escena desde el interfaz de CoppeliaSim (el fichero está en la raíz del repositorio).
2. Si modifica scripts Lua (por ejemplo `pioneer.lua`), asegúrese de copiarlos en la escena o en el script asociado de CoppeliaSim, ya que algunos cambios en el código fuente requieren pegar el script en la escena para que surtan efecto.
3. Instale Python y las dependencias tal y como se indica en A.1. (comando `pip install -r auxiliares/requirements.txt`).
4. Configure la ruta de CoppeliaSim en los scripts .bat si es necesario. Los ficheros `.bat` en la carpeta `scripts/` están preparados para lanzar CoppeliaSim y ejecutar los scripts de entrenamiento/evaluación; edítelos para adaptar la ruta si su instalación difiere.

A.3. Ejecución básica

- Ejecutar entrenamiento (modo por defecto):

```
```bash
python train.py
```
```

- `train.py` usa el entorno definido en `robot\_env.py` y guarda modelos y logs en el directorio `experiments/` (cada experimento tiene su propia carpeta con `model/` y `train\_logs/`).

- Evaluación rápida:

```
```bash
python evaluate.py
```
```

- Evaluación profunda (exporta CSV y figuras):

```
```bash
python evaluate_deep.py
```
```

- `evaluate\_deep.py` ejecuta la evaluación sobre un número grande de episodios (configurable) y guarda resultados en la carpeta `experiments/<exp\_name>/deep\_evaluation/`.

- Uso de los .bat (Windows): en `scripts/` hay atajos preparados (por ejemplo `runheadless.bat`, `evaluacion.bat`); edite las rutas internas si su instalación de CoppeliaSim no coincide con la ruta por defecto.

A.4. Reproducibilidad de experimentos

- Cada experimento se guarda bajo `experiments/exp\_xxx/` y contiene al menos: `model/` (modelo .zip), `train\_logs/` (TensorBoard y registros) y `deep\_evaluation/` (CSV y figuras). Para reproducir exactamente un experimento:

1. Coloque el modelo .zip en `experiments/<exp\_name>/model/` si no está ya.
2. Ejecute `python evaluate\_deep.py --exp <exp\_name>` o edite la constante de experimento en `evaluate\_deep.py` según su versión.

A.5. Solución de problemas comunes

- CoppeliaSim no responde o no conecta: compruebe que la API ZeroMQ está habilitada en la escena y que los puertos no están bloqueados por el firewall. Si usa la versión Edu, confirme que la licencia o limitaciones no impiden la ejecución headless.
- Error de importación de la librería ZeroMQ o cliente de CoppeliaSim: verifique `pip install -r auxiliares/requirements.txt` y, si es necesario, instale manualmente `pyzmq` y el cliente remoto.
- Permisos en Windows al lanzar CoppeliaSim: ejecute el .bat o CoppeliaSim como administrador si aparecen errores de acceso a puertos o a recursos.
- División por cero en cálculo de recompensas dispersas (durante los primeros pasos): el entorno incluye una protección que evita calcular proporciones hasta que exista al menos una visita registrada; si persiste el error, revise `robot\_env.py` donde se implementa la función de recompensa.

A.6. Ubicación de datos y resultados

- Modelos entrenados: `experiments/<exp\_name>/model/`
- Logs de entrenamiento (TensorBoard): `experiments/<exp\_name>/train\_logs/`
- Resultados de evaluación profunda y figuras: `experiments/<exp\_name>/deep\_evaluation/`

Apéndice B. Otros Recursos

Aquí se agrupan recursos, documentación y repositorios útiles para reproducir, ampliar o comprender los elementos técnicos del proyecto.

- **CoppeliaSim (simulación)**: Documentación oficial, descargas y ejemplos de escenas.

<https://www.coppeliarobotics.com>

- **ZeroMQ / API remota de CoppeliaSim**: Información sobre la API remota (ZeroMQ) usada para controlar el simulador desde Python y enlaces al cliente Python cuando esté disponible.

- Documentación remota de CoppeliaSim:
<https://www.coppeliarobotics.com/helpFiles/en/remoteApi.htm>

- PyZMQ (cliente ZeroMQ para Python): <https://pyzmq.readthedocs.io>

- **coppeliasim_zmqremoteapi_client**: Cliente Python usado en este proyecto (instalable vía pip).

Busque en PyPI o en el repositorio oficial de CoppeliaSim para la versión compatible.

- **Stable-Baselines3 (SB3)**: Implementaciones de algoritmos RL (incluye PPO y utilidades).

- Documentación: <https://stable-baselines3.readthedocs.io>

- Repositorio: <https://github.com/DLR-RM/stable-baselines3>

- **Gymnasium**: API y utilidades para entornos compatibles con OpenAI Gym (reemplazo/continuación).

<https://gymnasium.farama.org>

- **PyTorch**: Backend de cómputo usado por `stable-baselines3` en esta configuración.

<https://pytorch.org>

- **TensorBoard**: Visualización de métricas y curvas de entrenamiento.

<https://www.tensorflow.org/tensorboard>

- **Librerías de análisis y visualización**:

- NumPy: <https://numpy.org>
- Pandas: <https://pandas.pydata.org>
- Matplotlib: <https://matplotlib.org>
- tqdm (barras de progreso): <https://tqdm.github.io>
- ****Lecturas y tutoriales recomendados****:
 - Sutton, R. S. y Barto, A. G., "Reinforcement Learning: An Introduction" (online): <http://incompleteideas.net/book/the-book.html>
 - Schulman et al., "Proximal Policy Optimization Algorithms" (PPO): <https://arxiv.org/abs/1707.06347>
 - OpenAI Spinning Up (tutoriales prácticos de RL): <https://spinningup.openai.com>
- ****Repositorios y ejemplos prácticos****:
 - Ejemplos y scripts de SB3 (entrenamiento, evaluación, callbacks) en el repo de SB3.
 - Zonas de ejemplo de CoppeliaSim (escenas y scripts Lua) en la instalación de CoppeliaSim y en su repositorio de ejemplos.
- ****Herramientas adicionales útiles****:
 - Weights & Biases (seguimiento de experimentos): <https://wandb.ai>
 - Docker (contenedores reproducibles): <https://www.docker.com>
 - GitHub Actions (CI para reproducibilidad y tests): <https://docs.github.com/actions>
- ****Foros y soporte****:
 - Foro de CoppeliaSim y GitHub issues para problemas del simulador.
 - Issues y discusiones de `stable-baselines3` y `gymnasium` en GitHub.

Notas prácticas:

- Al instalar dependencias, prefiera entornos virtuales (`venv` o Conda) para evitar conflictos con paquetes del sistema.
- Si usa la versión Edu de CoppeliaSim en Windows, los scripts `.bat` en `scripts/` incluyen rutas por defecto que puede adaptar a su instalación: scripts.
- Para reproducir experimentos, siga las instrucciones de la sección de instalación y use los comandos de entrenamiento y evaluación indicados en los apartados correspondientes.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga